





Department of Computer Science

College of Computer Science and Information Systems

Najran University

Graduation Project Report

Artificial Intelligence Driven Drones For Border Security and Emergency Response in Rub' al-Khali

Prepared by:

Ghala Rashed Al-Dosari	443301403
Arwa Salem Al-Munajjim	443300632
Reham Taha Dabwan	443306645
Nouf Hamad Al-Ghubari	443301289
Dalal Hussain Al-Aqil	443301953

Supervisor: Dr. Saeed Saad Alahmari

2025 - 2026

Abstract

Monitoring large-scale border areas to detect infiltration and illegal activities, and conducting search and rescue operations in the desert to locate missing persons, are complex tasks that require accurate and reliable surveillance systems. In this work, we propose using drones equipped with infrared thermal imaging cameras as an effective solution for surveillance and search operations in low-visibility conditions, such as night and harsh weather. These systems enable detection based on thermal differences rather than visible light.

This work focuses on the automated detection of objects using static thermal images captured by drones to protect the borders of the Kingdom of Saudi Arabia and assist relevant authorities in expediting search operations for missing persons in the Rub' al Khali desert. The main objective is to distinguish between humans and animals under challenging conditions, including size differences, ambient thermal noise, limited visual detail, and varying camera angles, which pose significant challenges for traditional computer vision systems.

Due to the limited availability of realistic and adequately documented thermal datasets, synthetic thermal data generated using the Microsoft AirSim simulation platform is used to model realistic border environments with thermal camera configurations. The integration of real and artificial data contributes to improving the robustness of the deep learning model and its ability to generalize under realistic operational scenarios.

Table of Contents

Abstract ..... *I*

List of Figures ..... *V*

List of Tables ..... *VII*

List of Abbreviations ..... *VIII*

Chapter 1: Introduction *1*

1.1 Problem Statement **1**

1.2 Goals and Objectives **2**

1.2.1 Goals 2

1.2.2 Objectives 2

1.3 System Description **2**

1.4 Project Scope **3**

1.5 Project Plan **3**

1.5.1 Gantt Charts 5

Chapter 2: Deep Learning Background and Related Work *7*

2.1 Introduction **7**

2.2 Deep Learning: A Foundational Approach **7**

2.3 Convolutional Neural Networks (CNNs) for Thermal Object Detection **8**

2.3.1 General Architecture of a Convolutional Neural Network (CNN) 8

2.4 Overview of Relevent Architectures **9**

2.4.1 YOLO based object detection pipeline for human and animal detection in thermal drone images. 9

2.5 Thermal Imaging in Computer Vision **10**

2.6 Deep Learning Architectures for Human and Animal Detection **10**

2.7 Previous Work **10**

2.8 Summary **11**

Chapter 3: Dataset *12*

3.1 Dataset Overview **12**

3.2 Dataset Components **13**

3.2.1 Pre-existing Datasets 13

3.2.2 Synthetic Dataset 16

3.3 Data Cleaning and Preprocessing **19**

3.3.1 Cleaning the OSU Thermal Pedestrian Dataset 19

3.3.2 Cleaning the Terravic Motion IR Dataset 19

3.3.3 Cleaning and Converting the BirdsAi Dataset 20

3.3.4 Merging and Final Dataset Preparation 21

3.4 Labeling Process **22**

3.5 Dataset Statics and Visualization **25**

3.6 Dataset Limitation **26**

3.7 Dataset Summary 27
Chapter 4: Initial Experiment 31
4.1 Purpose of Initial Experiments 31
4.2 Dataset Distribution Summary 31
4.3 Preprocessing Steps 32
4.4 Baseline Model Selection 32
4.5 Software Libraries and Tools 33
4.5.1 TensorFlow / Keras 33
4.5.2 OpenCV (cv2) 33
4.5.3 NumPy 34
4.5.4 Matplotlib 34
4.5.5 Scikit-Learn 34
4.6 Training Configuration 35
4.7 Evaluation Metrics 35
4.8 Results 36
4.9 Discussion 39
4.9.1 Model Characteristics 40
4.9.2 Class Performance 40
4.9.3 Architectural Comparison 40
4.9.4 Future Enhancements 40
4.9.5 Practical Usage 40
4.10 Summary 41
4.10.1 Overall Performance 41
4.10.2 Model Strengths 41
Chapter 5: Implementation 42
5.1 Dataset Enhancement and Expansion 42
5.1.1 BirdsAI Dataset Expansion 42
5.2 Frame-Based Dataset Splitting 43
5.2.1 Background: How the Dataset Was Constructed 40
5.2.2 The Problem of Temporal Leakage 40
5.2.3 Detection and Confirmation of the Leakage Problem 40
5.2.4 Implementation of the Corrected Split 40
5.3 Enhanced Dataset Statics and Visualization 47
5.4 Last Yolo Model Training and Configurations 50
5.4.1 Model Selection: Yolo26m 50
5.4.2 Training Configuration 50
5.4 Final Model Evaluation 52
5.5.1 Evaluation Setup 52
5.5.2 overall Performance 52
5.5.3 Training Convergence 53
5.5.4 Class-wise Performance 54

Chapter 6: Summary and Future Work ***55***
6.1 Summary 55
6.2 Future work 56
Acknowledgement ***57***
References ***58***

List of Figures

Figure 1.1: Phase 1 Gantt Chart…….5

Figure 1.2: Phase 2 Gantt Chart…….6

Figure 2.1: An illustration of the relationship between AI, ML, DL, and CNN……7

Figure 2.2 : Architecture of Convolutional Neural Network (CNN)……8

Figure 2.3 : YOLO object detection architecture for thermal imagery…….9

Figure 3.1: Sample frames from the cleaned OSU Thermal Pedestrian Dataset……13

Figure 3.2: Example Terravic IR frames before and after cleaning……14

Figure 3.3: Examples of animal frame from the BirdsAI dataset with bounding boxes……15

Figure 3.4: Unreal Engine…….16

Figure 3.5: Synthetic Environment Sample with Wolfs……17

Figure 3.6: Synthetic Environment Sample with Sheeps……17

Figure 3.7: AirSim Drone Within Our Environment……18

Figure 3.8: MATLAB……..20

Figure 3.9: Python……...21

Figure 3.10: Label Studio……22

Figure 3.11: Labeling Settings……23

Figure 3.12: Real Images Labelling process……23

Figure 3.13: Synthetic Images Labelling process……24

Figure 3.14: Overall Image Distribution by Class……..25

Figure 3.15: Image Count per Class per Split…….25

Figure 3.16: Image Samples from each class……..26

Figure 4.1: Keras……33

Figure 4.2: TensorFlow……33

Figure 4.3: OpenCV……33

Figure 4.4: NumPy……..34

Figure 4.5: matplotlib……34

Figure 4.6: Scikitlearn…….34

Figure 4.7: Comparison of ResNet50 and MobileNet Performance Metrics……37

Figure 4.8: MobileNet Predictions …….38

Figure 4.9: ResNet50 Predictions……...39

Figure 5.1: Sample Frames and Split Distribution from the Terravic irw07 After Splitting……44

Figure 5.2: Overall Image Distribution (Animal vs Human) in Final Dataset……48

Figure 5.3: Image Count per Class per Split……...49

Figure 5.4: Confusion Matrix Showing Classification Results for Animal and Human…….53

Figure 5.5: YOLO detection results showing humans and animals with bounding boxes and confidence scores…….55

List of Tables

Table 1.1: Phase 1Project Plan…….3

Table 1.2: Phase 2 Project Plan…….4

Table 3.1: OTCBVS Benchmark Dataset Collection Summary……27

Table 3.2: BirdsAI Dataset Summary……..……28

Table 3.3: KSA SDS Dataset Summary……..30

Table 3.4: Real/Synthetic Summary…….……..30

Table 5.3: Final Dataset Split Summary…….47

Table 5.4: Image Count per Class per Split……47

Table 5.1: YOLO26m Training Hyperparameters……..51

Table 5.2: Performance Comparison on Validation and Test Datasets……52

List of Abbreviations

Abbreviation	Full Form
KSA SDS	KSA South Desert Simulation
YOLO	You Only Look Once
MOT	Multiple Object Tracking
UAV	Unmanned Aerial Vehicle
AI	Artificial intelligence
CNN	Convolutional Neural Network
TIR	Thermal Infrared
ML	Machine learning
DL	Deep learning
AI	Artificial intelligence
YOLOv3	You Only Look Once version 3
HOG	Histogram of Oriented Gradients
SVM	Support Vector Machine
LSTM	Long short-term Memory
Faster R-CNN	Faster Region-based Convolutional Neural Network

Chapter 1: Introduction

1.1 Problem Statement

Border security operations and search and rescue teams in the Empty Quarter face critical challenges in effectively monitoring the perimeter, particularly during nighttime and adverse weather conditions. Traditional camera-based surveillance systems suffer from significantly reduced visibility in low light environments, fog, rain, and dust storms, creating dangerous blind spots that compromise both security operations and rescue missions.

A major operational inefficiency stems from the high rate of false alarms caused by animal movements in the desert. Security personnel must respond to numerous false alarms, diverting resources from genuine threats and causing alert fatigue that can delay responses to actual infiltrations.

Existing fixed camera systems also offer limited coverage due to their fixed location, requiring extensive infrastructure and leaving gaps in surveillance coverage across vast border areas. Furthermore, while effective in the dark, thermal imaging cameras lack the intelligent discrimination capabilities to distinguish between humans and animals without human intervention.

This combination of environmental constraints, high false alarm rates, and inadequate automated threat classification creates an urgent need for a smart, mobile monitoring solution that can operate reliably in all conditions while accurately distinguishing between genuine security threats and benign wildlife activity.

1.2 Goals and Objectives

1.2.1 Goals

1. Enhancing border security and emergency response through Artificial intelligence AI powered UAVs drone surveillance.
2. Improving situational awareness in border areas, particularly under low visibility and nighttime conditions.
3. Reducing reliance on manual human patrols by providing automated and continuous border monitoring.
4. Reducing costs caused by false alarm detections.

1.2.2 Objectives

- To design, implement, and train a thermal based object detection model using UAV imagery.
- To distinguish humans from animals in thermal images captured by drone-mounted cameras.
- To utilize deep learning techniques for reliable object detection under low-visibility conditions.
- To generate and integrate synthetic thermal data using simulation tools to enhance training performance.
- To evaluate the system's performance using standard detection metrics.

1.3 System Description

The proposed system is an intelligent border surveillance solution that utilizes unmanned aerial vehicles equipped with thermal infrared cameras to support border security and emergency response. The system automatically detects living objects in thermal images and classifies them as humans or animals using deep learning-based object detection techniques.

Thermal images captured by the drone are processed to identify thermal patterns under low-visibility and nighttime conditions. By relying on automated detection rather than manual monitoring, the system enhances situational awareness and enables continuous and reliable border surveillance.

1.4 Project Scope

This project focuses on the development of a deep learning-based object detection system for border surveillance using thermal images captured by unmanned aerial vehicles (UAV). The scope of the project is limited to detecting and classifying living objects into two categories: human and animal.

1.5 Project Plan

The project team communicates with the supervisor through regular meetings held on Google Meet two days per week. Team members also communicate with each other using WhatsApp and meet through Google Meet or at the university when needed to discuss ideas conduct research and analyze data. The supervisor assigns tasks to the team through Notion according to the approved project schedule. After completing each task the work is uploaded to a shared Google Drive for review and documentation. The tasks for each phase of the project are presented in the tables below.

Week(s)	Task
1-2	Studying the proposed ideas
3-4	Reading research papers
5	Dataset collection, preparation, and cleaning
6	BirdsAI dataset to YOLO and verification.
7-8-9	Synthetic dataset generation using Microsoft AirSim
10	Deep learning methods (summary)
11	Initial experiment
12	Writing report
13	Preparing presentation
14	Project presentation for the committee

Table 1.1: Phase 1Project Plan

Week(s)	Task
1-2	Expanding our dataset
3-4-5-6-7	YOLO model
8-9	Writing the report
10	Publishing the code to GitHub
11	Writing the research paper
12	Submitting the paper
13	Preparing the presentation
14	Project presentation for the committee

Table 1.2: Phase 2 Project Plan

1.5.1 Gantt Charts

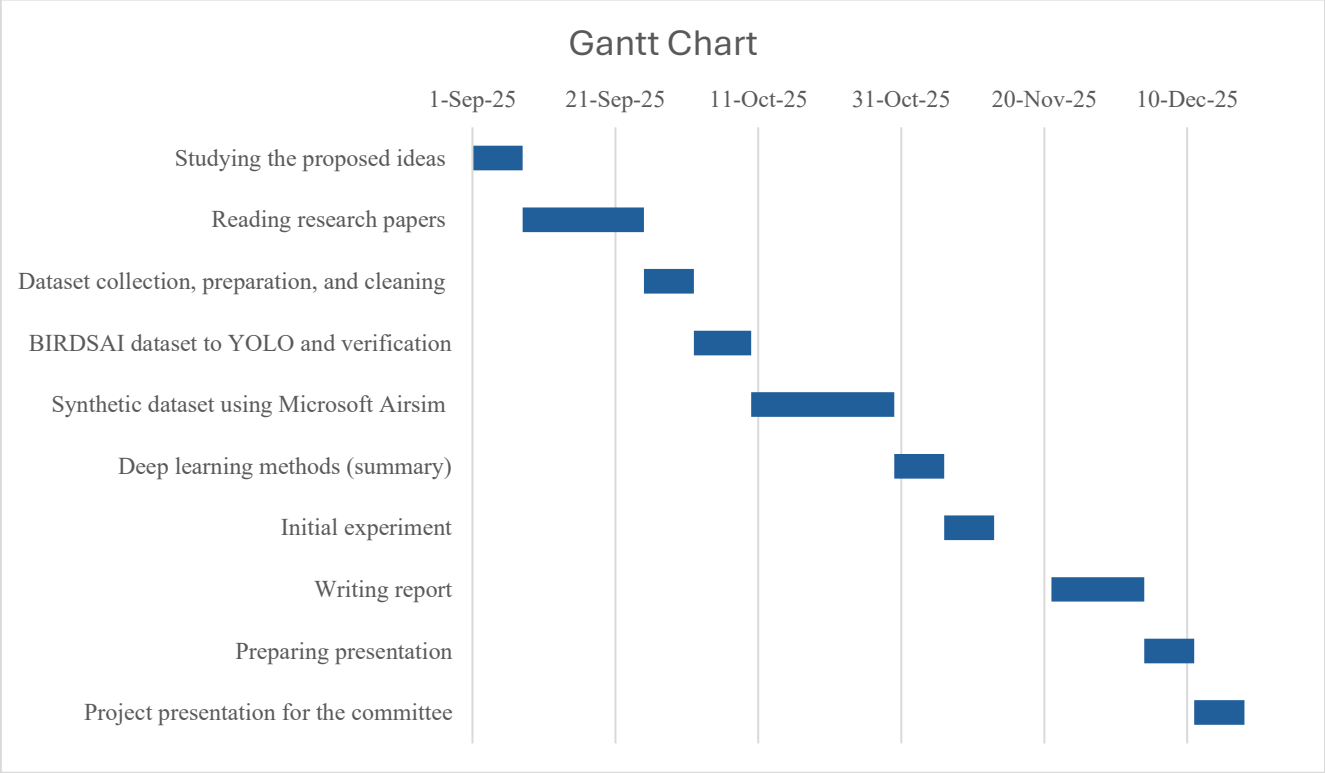


Figure 1.1: Phase 1 Gantt Chart

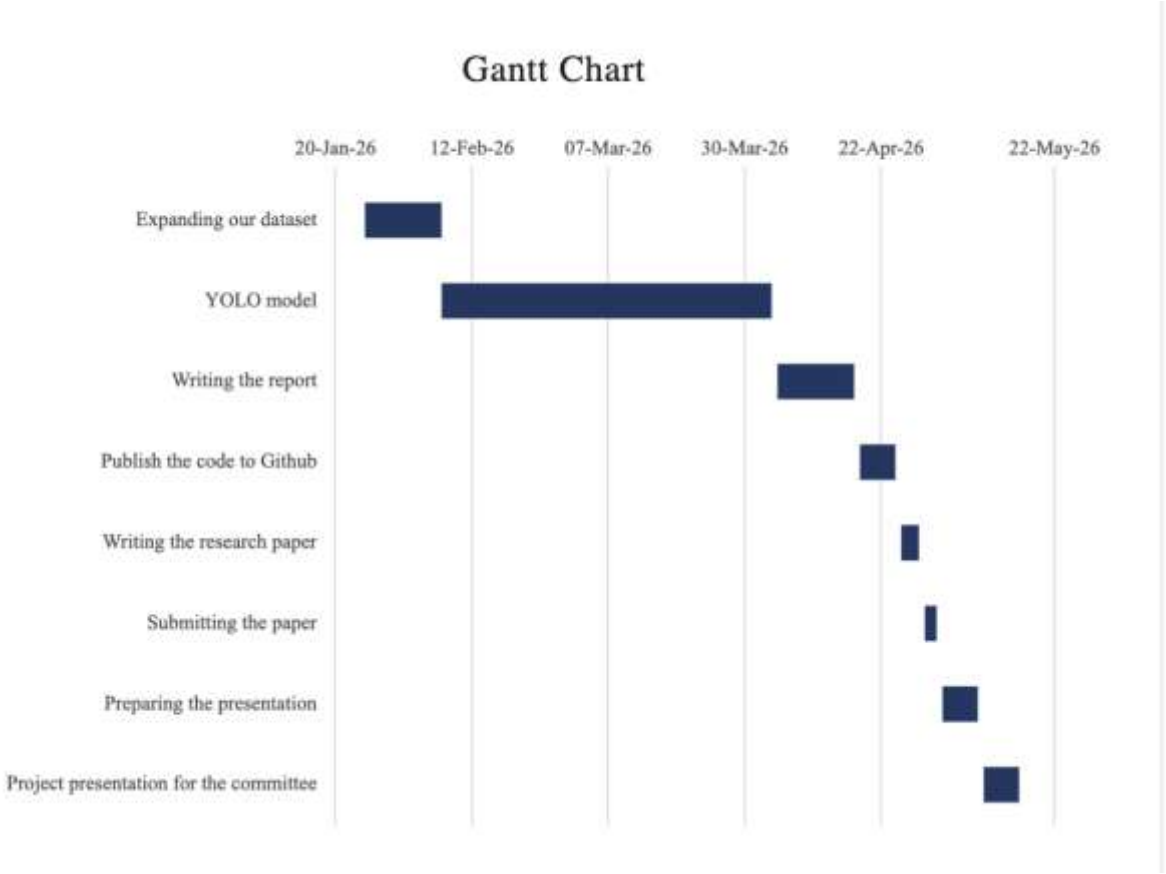


Figure 1.2: Phase 2 Gantt Chart

Chapter 2: Deep Learning Background and Related Work

2.1 Introduction

This chapter provides an overview of the deep learning concepts and architectures that support the development of our thermal-based object detection system. The project relies on detecting and locating humans and animals in thermal imagery captured by a drone, making deep learning particularly CNN based detectors an essential component. In this chapter, we discuss key architectures, thermal imaging fundamentals, and relevant research that aligns with the objectives of our system.

2.2 Deep Learning: A Foundational Approach

Deep learning enables models to learn useful patterns directly from raw thermal images without manual feature design. This is especially important because thermal data often lacks clear texture and visual detail. Convolutional Neural Networks (CNNs) are the most effective for this setting, as their layered filters can capture temperature variations, shapes, and silhouettes even under low contrast. Architectures like YOLO and ResNet rely on these properties to achieve strong performance in object detection tasks.

Although recurrent models are useful for sequential analysis, our project focuses on frame-based detection, where CNNs offer the most efficient and reliable solution for distinguishing humans from animals in aerial thermal imagery.

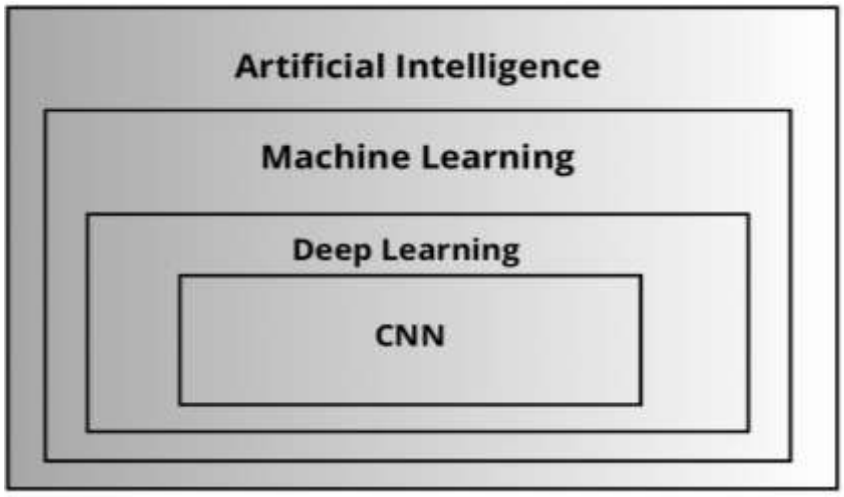


Figure 2.1: An illustration of the relationship between AI, ML, DL, and CNN

2.3 Convolutional Neural Networks (CNNs) for Thermal Object Detection

Convolutional Neural Networks (CNNs) are widely used for thermal image analysis because they learn spatial hierarchies and capture heat-based patterns that remain reliable even when lighting conditions vary. In thermal object detection (Mittal et al., 2019)[1], CNNs can distinguish signatures emitted by humans, vehicles, and other heat-generating targets.

Architectures such as Faster R-CNN and YOLO rely on convolutional layers to extract deep thermal features, enabling robust detection even under low contrast or challenging nighttime environments where visible-spectrum cameras typically fail.

In our system, CNN-based detectors form the core of the pipeline, allowing the drone to accurately locate humans and animals within each thermal frame—consistent with the study's findings that thermal images achieve higher recall and accuracy than visible images.

Additionally, incorporating data augmentation and exposure to varied thermal conditions improves model robustness against differences in scale, temperature, and environmental settings

2.3.1 General Architecture of a Convolutional Neural Network (CNN)

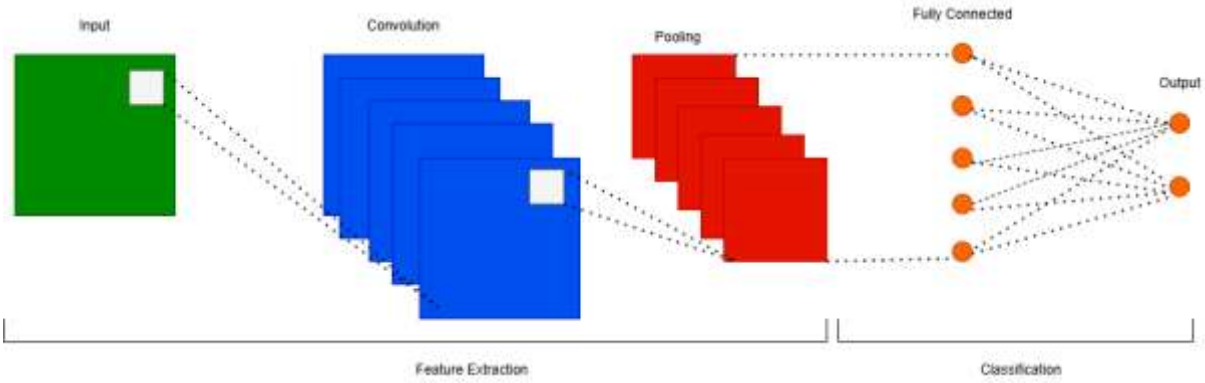


Figure 2.2 : Architecture of Convolutional Neural Network (CNN) [2]

Figure 2.2 illustrates the general architecture of a Convolutional Neural Network (CNN) used for image classification tasks. The process begins with an input image that is passed through multiple convolutional layers to extract relevant features such as edges, shapes, and patterns. These features are then refined through pooling layers, which reduce the spatial dimensions while preserving important information. Finally, the extracted features are fed into fully connected layers that perform classification and generate the final output.

2.4 Overview of Relevent Architectures

Deep learning architectures used in object detection vary in their design and feature extraction capabilities, with modern models focusing on improving accuracy, robustness, and efficiency. Architectures such as YOLO, ResNet, EfficientNet, and Vision Transformers (ViT) introduce strong representational power that helps models handle challenges common in thermal imagery, including low contrast, weak textures, and scale variation.
Modern architectures including **ResNet, EfficientNet**, and **Vision Transformers (ViT)** introduce advanced feature extraction capabilities. These models help address challenges often present in thermal imagery such as low contrast, weak textures, and scale variation.

For our project, YOLO-based detectors are especially important because of their fast inference speed, which enables the drone to detect humans and animals in real time during flight. In parallel, deeper networks like ResNet contribute to stronger feature extraction from thermal frames, improving the system's ability to differentiate between human and animal heat signatures.

2.4.1 YOLO based object detection pipeline for human and animal detection in thermal drone images.

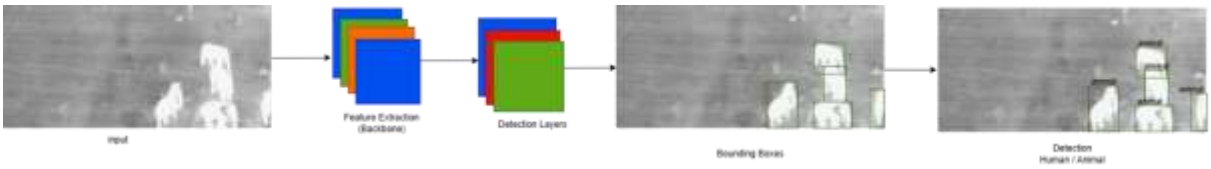


Figure 2.3 : YOLO object detection architecture for thermal imagery.

In the proposed system, YOLO is used as the core object detection model to identify and classify humans and animals in thermal drone images. The system operates as follows:

- **Input Image:**
  The process begins with a thermal image captured by a drone-mounted thermal camera.
- **Feature Extraction (Backbone):**
  The backbone network applies multiple convolutional layers to extract important features from the input image, such as shapes, edges, and thermal patterns.
- **Detection Layers:**
  The extracted features are processed through detection layers that prepare the feature maps for object localization and classification.
- **Bounding Boxes:**
  The model predicts bounding boxes that indicate the spatial locations of detected objects within the image.
- **Final Detection (Human / Animal):**
  Each detected object is classified as either a human or an animal, along with an associated confidence score.

2.5 Thermal Imaging in Computer Vision

Thermal imaging captures infrared radiation emitted by warm-blooded objects, making it effective for nighttime and low-visibility monitoring. Compared to RGB cameras, thermal sensors provide advantages such as:

- Robustness against lighting changes
- The ability to detect humans and animals even in complete darkness
- Reliable performance in fog, dust, and harsh desert weather
- No dependency on external illumination sources

However, thermal images suffer from low spatial resolution, limited texture, and environmental temperature variations.
For our project, thermal imaging allows the drone to operate in remote border regions regardless of lighting conditions, enabling continuous monitoring where traditional surveillance systems fail.

2.6 Deep Learning Architectures for Human and Animal Detection

Detecting humans and animals in thermal imagery is challenging due to similar heat signatures. Deep learning detectors trained on thermal datasets learn subtle differences in heat distribution and shape. Lightweight models like YOLO allow **real-time detection**, while deeper architectures like ResNet and DenseNet improve **bounding-box accuracy**. These approaches support building a reliable system for distinguishing objects in desert border environments[3].

2.7 Previous Work

Thermal UAV datasets remain limited, which restricts the development of robust detection systems. The most relevant work to our project is (Bondi et al., 2020) [4], which introduced 48 real UAV thermal videos and 124 synthetic sequences generated using AirSim. The dataset addresses key challenges such as motion blur, scale variation, and weak thermal texture, and demonstrates that combining real and synthetic data improves model generalization, an approach closely aligned with our system design. Several studies have explored the use of deep learning for detecting animals in thermal imagery. For example, the paper (Murugesan & Mahendiran, 2024) [5] compared YOLOv3, Faster R-CNN, and traditional methods such as HOG/SVM. The results showed that deep learning models significantly outperform conventional thermal-processing techniques, supporting our decision to frame the task as object detection rather than simple classification. Other research has incorporated temporal information into thermal drone surveys. The paper (Backman et al., 2025) [6] integrates YOLO with LSTM layers to leverage motion cues, improving the distinction between terrestrial and arboreal animals. Although our system uses frame-based detection, this work highlights the importance of varied environmental conditions and realistic motion patterns, principles reflected in the diverse synthetic environments we built in AirSim. Several works have introduced methods to enhance thermal imagery and improve deep model robustness. Fusion-based approaches such as (IPPR, 2024) [7], (Wang et al., 2025) [8], and (Xue et al., 2023) [9] employ attention mechanisms to handle issues

like occlusion, illumination changes, and thermal crossover. Additionally, the study (Tan et al., 2022) [10] demonstrated that contrast enhancement significantly boosts YOLO detection performance. These findings informed our preprocessing strategy for thermal normalization and contrast adjustment.

While most prior thermal UAV studies primarily address animal detection, our project extends this scope by targeting binary human–animal detection as a unified task.

Collectively, these studies highlight meaningful progress but also reveal persistent limitations in thermal UAV detection particularly for binary human-animal scenarios-setting the stage for the following contribution.

Despite progress in thermal detection, existing literature lacks a dedicated UAV framework focused specifically on **binary human–animal detection** in desert and mountainous border-like conditions.Additionally, our work develops a dedicated deep learning model tailored for human detection in thermal imagery, addressing a gap not explicitly tackled in prior UAV-focused research.

Our project addresses this gap by:

- Designing **custom synthetic thermal scenes** in AirSim that reflect real regional terrain.
- Combining **real and synthetic thermal data** to strengthen model robustness.
- Creating **high-precision manual annotations** using Label Studio.

This positions our system as a domain-focused extension to existing thermal UAV research.

2.8 Summray

This chapter outlined the theoretical foundations and prior research relevant to thermal drone-based object detection. We reviewed key deep learning concepts, characteristics of thermal imagery, and recent advancements in aerial TIR datasets and wildlife detection methods. The literature consistently highlights challenges such as low resolution, weak texture, and environmental variability—factors that directly influence detection performance.

The insights from previous work justify our focus on combining synthetic and real thermal data, adopting robust CNN-based detectors, and designing environment-specific training scenarios. These findings provide the conceptual basis for the methodology presented in the next chapter.

Chapter 3: Dataset

3.1 Dataset Overview

The dataset used in our project was constructed from multiple sources, each contributing different types of thermal imagery, environments, and annotation formats. Because the final goal of the project is to develop an object detection model capable of identifying humans and animals in thermal drone imagery, the dataset needed to be both diverse and representative of real-world conditions, such as low visibility, motion, environmental noise, and changing temperature patterns.

To achieve this, the dataset was built from three main components:

1. **Public thermal datasets** from *the OTCBVS Benchmark Collection***[11]** :
   i. Ohio State University (OSU) Thermal Pedestrian Dataset
   ii. Terravic Motion IR Dataset
2. **Aerial dataset (BirdsAI) [4]** originally annotated in the MOT format.
3. **A synthetic desert dataset (KSA SDS)** that we generated using Unreal Engine + AirSim to simulate realistic wildlife and human thermal signatures in Saudi Arabian desert environments.

Each dataset had a different structure, image quality, and annotation format, which required significant cleaning and preprocessing before integration into a unified dataset suitable for deep learning.

The main goal of combining these datasets was to build a large and diverse thermal-image collection that represents a wide range of environments (indoor, outdoor, desert, and aerial views), different object types (humans and various animals), and varying thermal conditions such as night scenes, low contrast, noise, and partial occlusion. The datasets also offered multiple viewpoints, including drone footage, ground cameras, static surveillance systems, and simulated drone environments, which helped make the final dataset more robust. During dataset construction, several challenges appeared consistently across all sources, such as the presence of large amounts of low-quality or unusable thermal images with blur or overheating artifacts, inconsistent labeling where some objects were poorly annotated or not annotated at all, and strong class imbalance since most publicly available thermal datasets only contained humans. These challenges required a combination of manual cleaning, preprocessing, and selective synthetic data generation to produce a dataset suitable for training deep learning models.

3.2 Dataset Components

The final dataset has many datasets each with unique characteristics, environments, and labeling formats. In the following subsections, we describe each dataset, focusing on its origin, quality, annotation issues, and what modifications were required before merging it with the rest of the dataset.

3.2.1. Pre-existing Datasets

Ohio State University (OSU) Thermal Pedestrian Dataset

The OSU Thermal Pedestrian Dataset is one of the earliest publicly available thermal datasets and consists mainly of pedestrians captured using a static thermal camera.

Even though the dataset is old, it was still useful because it provided clear human thermal signatures that helped establish a baseline understanding of thermal patterns and temperature gradients. However, a significant portion of the frames contained noise, shaking, or low dynamic range, so we manually removed the unusable images and kept only the clean sequences that clearly showed human silhouettes.

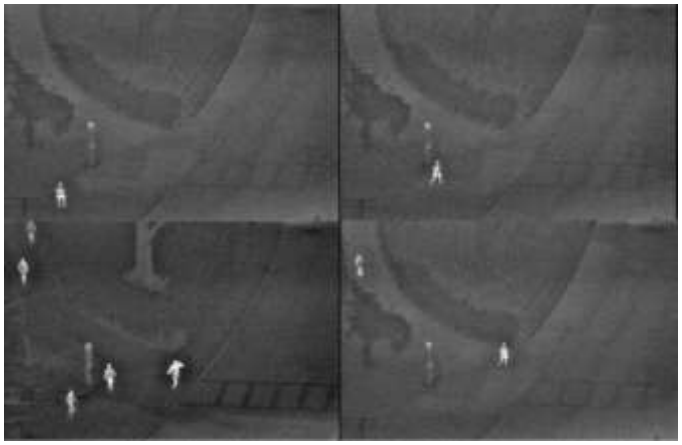


Figure 3.1: Sample frames from the cleaned OSU Thermal Pedestrian Dataset

Terravic Motion IR Dataset

The Terravic Motion IR Database is part of the OTCBVS Benchmark Dataset Collection and contains multiple thermal video sequences captured in different environments. The dataset is organized into six main folders, but only one of them was suitable for our use case. The “Indoor Hallway Surveillance” folder was removed because all the footage was recorded indoors, which does not match the intended drone-based outdoor detection scenario. The “Outdoor House Surveillance” folder was also deleted since it only contained static scenes without humans or animals. From the “Outdoor Motion and Tracking Scenarios” folder, 10 out of 11 sequences were useful and kept, while the last sequence was removed due to low relevance. The remaining three folders— “Plane Motion and Tracking,” “Underwater and Near-Surface Motion,” and “Uneventful Background Motion”—were all excluded because they either lacked humans or animals or contained unclear footage that would not contribute to model performance. After this cleaning step, only the high-quality, relevant outdoor motion sequences were kept and later annotated for the human and animal classes used in this project.

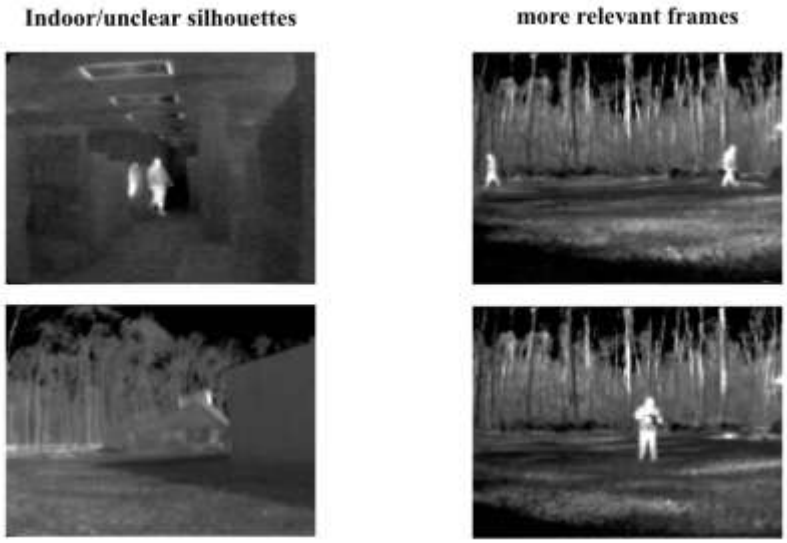


Figure 3.2: Example Terravic IR frames before and after cleaning

BirdsAI Dataset

The BirdsAI dataset was originally created for aerial wildlife monitoring, and it contains thermal footage of various animal species captured from drones. Unlike the OSU and Terravic datasets, BirdsAI images were annotated in the MOT (Multiple Object Tracking) format, which is not directly compatible with object detection frameworks such as YOLO. To prepare this dataset, we converted the MOT labels into YOLO annotation and removed frames where the animals were too small or impossible to identify.

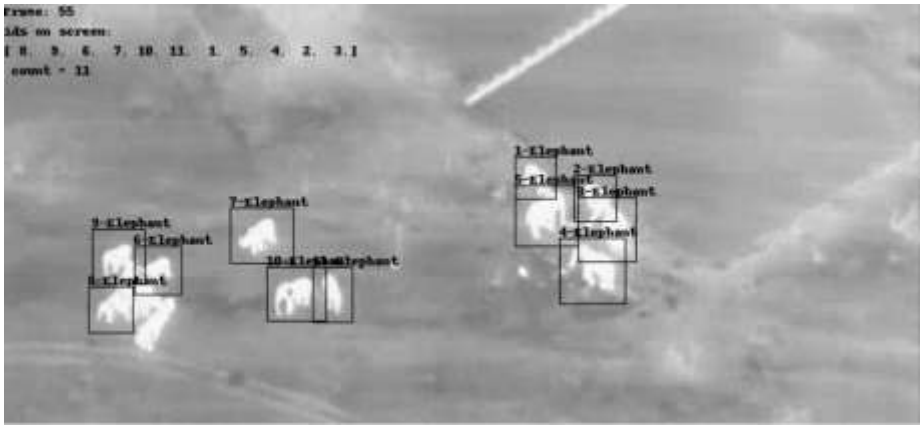


Figure 3.3: Examples of animal frame from the BirdsAI dataset with bounding boxes

Because this dataset was the only real-world source containing animals, it played a key role in balancing the human–animal class distribution during the early experiments.

3.2.2. Synthetic Dataset

The **KSA SouthDesertSim (KSA SDS) dataset** was developed to overcome a key limitation in existing thermal-image datasets: the lack of high-quality thermal data representing the Saudi Arabian desert environment, specifically the southern border region. To address this gap, a custom synthetic dataset was generated using Unreal Engine and AirSim**)[12]**, combining realistic 3D modeling with physics-accurate drone simulation.

Unreal Engine Environment Simulation

Unreal Engine was used to construct a highly realistic desert environment. The terrain was modeled to reflect the topography of southern Saudi Arabia, including sand dunes, rocky areas, dry shrubs, and open flat regions. High-quality 3D animal and human models were placed in the environment, each animated with natural movements such as walking, running, grazing, or standing.

To ensure realism in thermal imaging, custom shaders were applied to models to simulate thermal emissivity, giving different materials (skin, fur, fabric) the proper heat signature. Environmental lighting and temperature variations were also adjusted to reflect daytime and nighttime desert conditions.



Figure 3.4: Unreal Engine



Figure 3.5: Synthetic Environment Sample with Wolfs



Figure 3.6: Synthetic Environment Sample with Sheeps

AirSim for Drone Simulation and Thermal Capture

The synthetic images were captured using **AirSim [12]**, an open-source simulator for drones and autonomous vehicles developed by Microsoft Research.

Python scripts were used to control drone missions, specifying altitude, tilt angle, camera FOV, and capture frequency. This allowed the dataset to include a wide range of conditions.



Figure 3.7: AirSim Drone Within Our Environment

3.3 Data Cleaning and Preprocessing

Data cleaning was the most time-consuming part of the project because the datasets came from very different sources, with different formats, annotation styles, image quality levels.

The goal of this stage was to remove noisy or unusable frames, ensuring consistent annotation formats, and prepare a clean and unified dataset that could be used later in the project.

3.3.1 Cleaning the Ohi State University (OSU) Thermal Pedestrian Dataset

This dataset originally contained 284 thermal images. However, most frames were low-resolution, noisy, blurred, or missing at least 50% of the human body.

After filtering: Only 72 images were kept.

3.3.2 Cleaning the Terravic Motion IR Dataset

The Terravic dataset required the most extensive cleaning. Although it contains thousands of thermal frames, many of them do not include any humans or animals, and several folders are completely irrelevant for the project.

Folder Filtering

The dataset contains six main categories. Only **Outdoor Motion and Tracking Scenarios** was relevant. The following folders were deleted entirely:

- Indoor Hallway Surveillance (indoor only, no relevance)
- Outdoor House Surveillance (house only, no clear humans/animals)
- Plane Motion & Tracking (no humans or animals)
- Underwater & Near-Surface Motion (unclear/unusable)
- Uneventful Background Motion (only clouds/trees)

Frame-Level Cleaning

For each of the remaining outdoor collections (irw01–irw10), every frame was manually checked and filtered. Frames were deleted for the following reasons:

- No human/animal present
- Low quality or unclear shapes
- Motion blur or thermal distortion
- Severe occlusion

This reduced the dataset significantly. For example:

- irw01: from 650 → 470 images
- irw07: from 1300 → 1067 images
- irw09: from 1620 → only 279 usable images

3.3.3 Cleaning and Converting the BirdsAi Dataset

The BirdsAI dataset came with MOT-format annotation, which are not compatible with object detection frameworks such as YOLO. The cleaning process included:

Step 1: Frame Filtering

Many frames were removed because they contained:

- No humans or animals
- Objects too small to be detected
- Noise

Step 2: Annotation Conversion

This step was done using MATLAB script that reads each MOT annotation from .cvs file, extracts the bounding -box coordinates *(x, y, width, height),* and transforms them into YOLO format *(class, x_center, y_center, width, height)*. MATLAB was used because it made it easy to handle large numeric matrices, loop through thousands of annotation rows, and apply coordinate normalization automatically for each frame's resolution. During this conversion step, tracking IDs were removed since they are not required for object detection, and all species labels were mapped into only two classes: Animal (0) and Human (1). The final output was a clean set of YOLO-ready annotation files.



Figure 3.8: MATLAB

3.3.4 Merging and Final Dataset Preparation

To complete the dataset preparation process, we used Python scripts that automatically merged all cleaned datasets into a single unified structure and then generated the final Train/Validation/Test split. The first script scans every folder inside the project directory and looks for valid image–label pairs. Because different sources used different naming conventions (e.g., “images”, “images_renamed”), the script checks both options and copies only valid files into a common folder. During this step, each file is renamed using the pattern foldername_filename to avoid any naming conflicts when merging thousands of images from different datasets. The second Python script takes the merged dataset and splits it into 70% training, 15% validation, and 15% testing, using a random shuffle to ensure a balanced distribution. Both the images and their corresponding labels are copied into YOLO-style directories (images/train, labels/train, etc.), guaranteeing that every sample stays paired correctly. This automated pipeline prevented human error, kept the dataset consistent, and made the entire merging and splitting process fully reproducible.



Figure 3.9: Python

3.4 Labeling Process

Since both datasets used required manual labeling, a dedicated labeling workflow was implemented to ensure accurate and consistent annotations. The Terravic Motion IR dataset did not include any bounding-box labels, so every usable frame had to be labeled from scratch. The OSU Thermal Pedestrian dataset, although it contained ground-truth bounding boxes, proved to have many unreliable annotations (e.g., boxes where less than 50% of the person was visible, incorrect box coordinates, and many low-quality frames). These wrong annotations were identified, corrected, or removed during our labeling pass. For BirdsAI, the provided MOT tracking annotations were converted to detection format and then verified and corrected where necessary.

To manage all labeling and corrections efficiently, we used Label Studio as the main annotation tool because it supports multi-class boxes, and direct export to YOLO format. The labeling setup included two classes Human and Animal and every retained frame was inspected to confirm object visibility, remove ambiguous samples, and ensure that bounding boxes matched the actual infrared silhouettes. After manual annotation and correction, annotations were exported in YOLO format for integration with the training pipeline. This careful process was essential because thermal images often include low contrast, partial visibility, and noise, which require human judgment to annotate correctly.



Figure 3.10: Label Studio

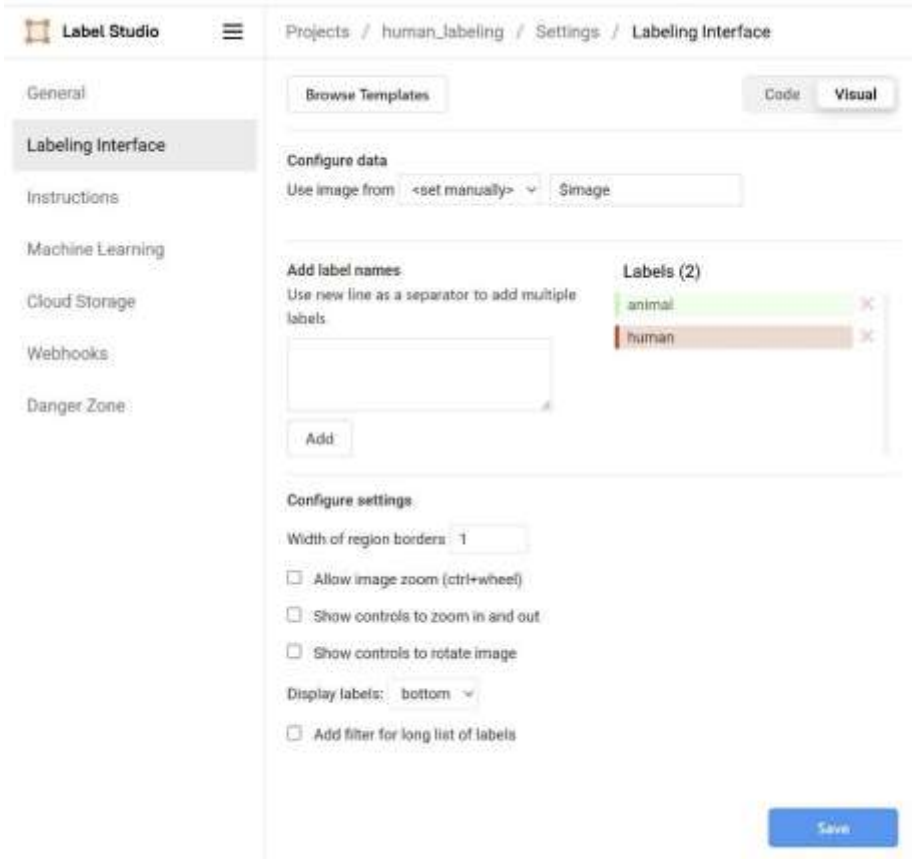


Figure 3.11: Labeling Settings

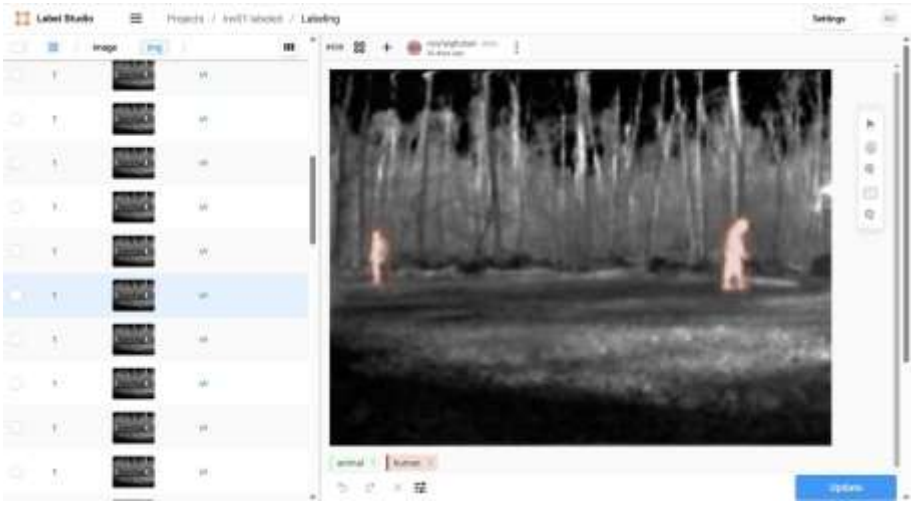


Figure 3.12: Real Images Labelling process



Figure 3.13: Synthatic Images Labelling process

3.5 Dataset Statics and Visualization

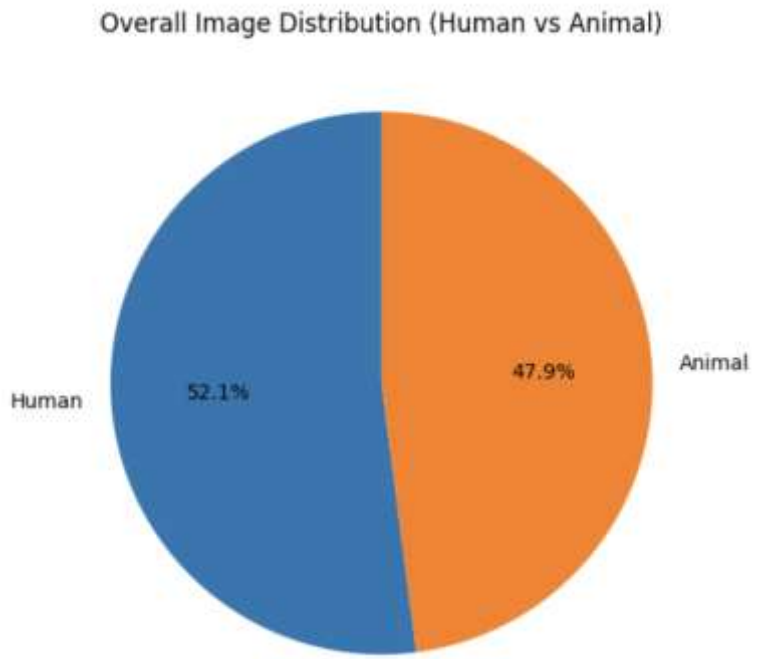


Figure 3.14: Overall Image Distribution by Class

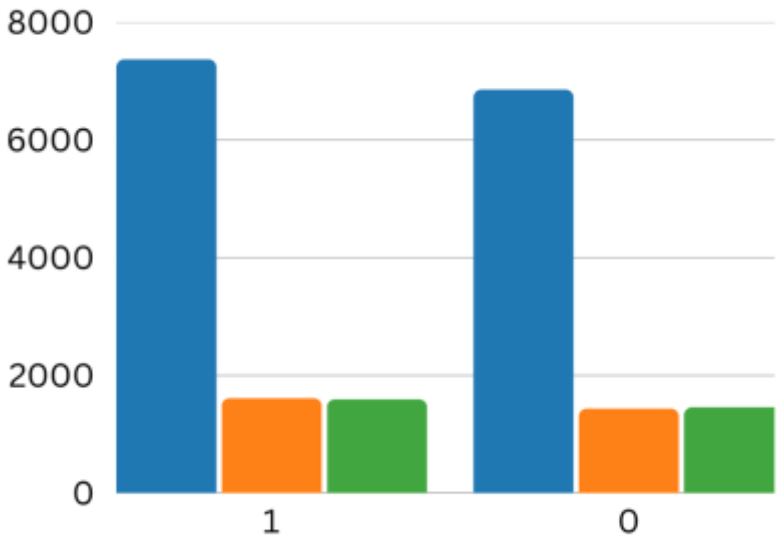


Figure 3.15: Image Count per Class per Split

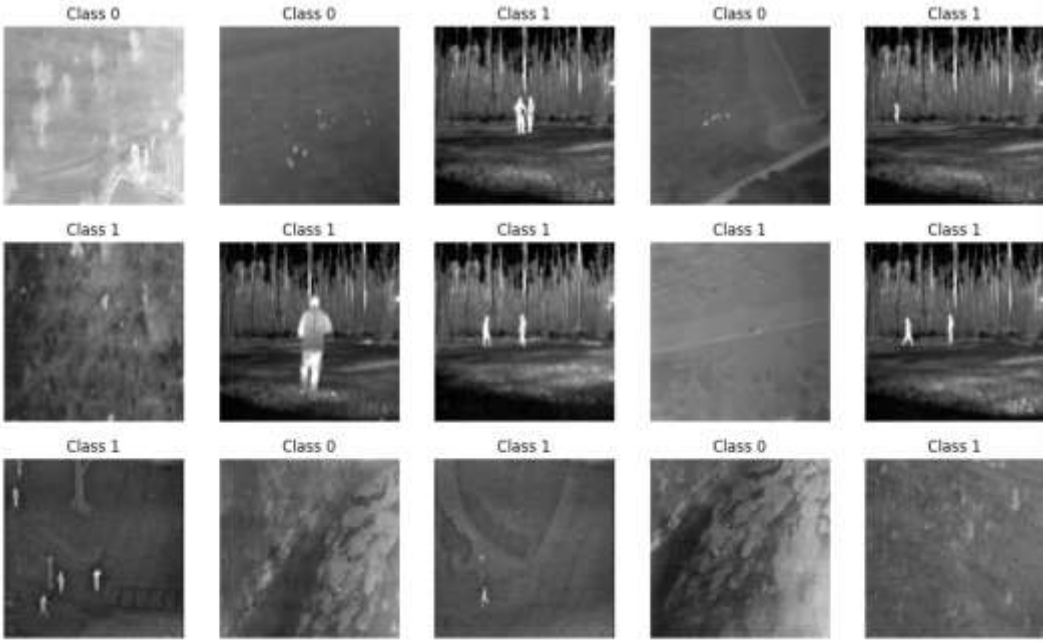


Figure 3.16: Image Samples from each class

3.6 Dataset Limitation

The dataset is generally balanced and diverse in terms of classes and image conditions. However, it lacks real images from Saudi desert environments. As a result, the trained model may not fully generalize to desert-specific scenarios, which could limit its performance in such contexts.

3.7 Dataset Summary

Collection name	Number of Images (Animals)	Number of Images (Human)	Number of Images (deleted)	Reason
#1, 1~10	0	72	212	Low quality
#5, irw1.zip	0	470	180	No animal/human present
#5, irw2.zip	0	689	171	No animal/human present
#5,irw3.zip	0	611	189	No animal/human present
#5,irw4.zip	0	708	192	No animal/human present Occlusion Unclear object
#5,irw5.zip	0	678	172	No animal/human present
#5,irw6.zip	0	1028	177	No animal/human present Occlusion Unclear object
#5,irw7.zip	0	1067	233	No animal/human present, noisiness, unclearness
#5,irw8.zip	0	154	206	No animal/human present
#5, irw9.zip	0	279	1341	No animal/human present, noisiness, unclearness
#5,irw10.zip	0	257	234	No animal/human present
Total	**0**	**6013**	**3307**	

Table 3.1: OTCBVS Benchmark Dataset Collection Summary

Frame#	Number of Images (Animals)	Number of Images (Human)	Number of Images (deleted)	Reason
67_14	250	0	710	No animal/human present Occlusion Unclear object
67_19	260	0	710	No animal/human present Occlusion Unclear object
67_26	264	0	704	No animal/human present Occlusion Unclear object
67_27	243	0	761	No animal/human present Occlusion Unclear object
67_28	159	0	763	No animal/human present Occlusion Unclear object
67_29	67	0	889	No animal/human present Occlusion Unclear object
67_40	590	0	413	No animal/human present Occlusion Unclear object
67_41	665	0	310	No animal/human present Occlusion Unclear object

67_45	151	0	761	No animal/human present Occlusion Unclear object
67_46	234	0	716	No animal/human present Occlusion Unclear object
67_50	0	914	42	No animal/human present Occlusion Unclear object
67_52	0	331	538	No animal/human present Occlusion Unclear object
67_55	334	0	562	No animal/human present Occlusion Unclear object
67_58	657	0	311	No animal/human present Occlusion Unclear object
67_59	428	40	486	No animal/human present Occlusion Unclear object
351	142	0	0	No animal/human present Occlusion Unclear object
359	0	232	3	No animal/human present
363	0	1213	1387	No animal/human present Occlusion Unclear object

364	0	2066	3276	No animal/human present Occlusion Unclear object
367	5087	0	361	No animal/human present Occlusion Unclear object
Total	**9531**	**4796**	**13703**	

Table 3.2: BirdsAI Dataset Summary

(KSA SDS)	**Number of Images**
Camels	223
Deers & Does	195
Zebra	200
Fox	200
Wolf	196
Sheeps	358
Dog	187
Cat	203
Human	1612

Table 3.3: KSA SDS Dataset Summary

Images	**Real**	**Synthetic**
Number of images	20340	3374

Table 3.4: Real/Synthetic Summary

Chapter 4: Classification Experiment

4.1 Purpose of Initial Experiments

The initial experiments aim to establish baseline performance for two widely used CNN architectures, ResNet50**[13]** and MobileNet**[14]**, on the merged dataset before exploring more advanced or customized models. The objectives include:

• Assessing the ability of standard CNNs to classify objects into **Animal (Class 0)** and **Human**

(Class 1).

• Providing insights for model selection, preprocessing, and training configuration in subsequent experiments.

• Comparing lightweight (MobileNet) and deep (ResNet50) architectures to determine a suitable starting point for the project.

4.2 Dataset Distribution Summary

The real dataset, after merging and preprocessing, was split into training (70%), validation (15%), and testing (15%) subsets. It contains 79,466 annotated objects across two classes, with all dataset preparation steps carried out using Python code:

Subset	Animal (0)	Human (1)	Total
Training	45,062	11,131	56,193
Validation	9,124	2,363	11,487
Testing	9,734	2,352	12,086

Both models were trained and evaluated on the same dataset splits, ensuring a fair comparison of performance.

4.3 Preprocessing Steps

For both models, the preprocessing pipeline included:

Resizing to 224×224 pixels to fit CNN input requirements.

Normalization: Pixel values scaled to [0, 1].

Label Conversion: YOLO-style annotations were retained as integer class labels (0 for Animal, 1 for Human) and directly used for batch generation during training .

Batch Generation: Custom YOLO Generator was implemented to produce batches during training.

No data augmentation was applied at this stage to measure baseline capabilities of each architecture.

4.4 Baseline Model Selection

Two models were chosen as baselines:

1. **ResNet50:**
    - Deep residual network suitable for complex feature extraction.
    - Pre-trained on ImageNet and fine-tuned on the dataset.
    - Entire network set as trainable.
2. **MobileNet:**
    - Lightweight CNN optimized for faster inference.
    - Pre-trained on ImageNet **[15]** and fine-tuned similarly.
    - Suitable for deployment on resource-limited environments.

These selections allow a comparative analysis between a deep heavyweight model and a lightweight efficient model.

4.5 Software Libraries and Tools

4.5.1 TensorFlow / Keras

These libraries were essential for building, training, and fine-tuning the ResNet50 and MobileNet models. Keras provided a high-level interface for designing the neural network architectures, while TensorFlow handled backend computation and GPU acceleration.



Figure 4.1: Keras



Figure 4.2: TensorFlow

4.5.2 OpenCV (cv2)

OpenCV was used for image loading, resizing, and processing YOLO-style annotations. It supported key operations in dataset preparation before feeding images into the deep learning models.



Figure 4.3: OpenCV

4.5.3 NumPy

NumPy served as the foundation for numerical operations throughout the project. It was used for manipulating arrays, managing batches, and ensuring efficient preprocessing workflows.



Figure 4.4: NumPy

4.5.4 Matplotlib

Matplotlib was used to generate visualizations such as training history curves and dataset distribution charts. These plots supported performance analysis and model comparison.



Figure 4.5: matplotlib

4.5.5 Scikit-Learn

Scikit-Learn provided the evaluation tools required to compute key performance metrics, including Accuracy, Precision, Recall, and F1-Score, enabling a standardized assessment of the model results.



Figure 4.6: Scikitlearn

4.6 Training Configuration

Both models (MobileNet and ResNet50) were trained under comparable settings to ensure a fair baseline evaluation, with controlled variations based on the requirements of each architecture. A batch size of 32 and a total of 50 training epochs were applied in both experiments, and the models were trained using categorical cross-entropy as the loss function, reflecting the two-class (animal vs. human) classification setup. Training was conducted on Google Colab with GPU acceleration.

The Adam optimizer was used for both models, with learning rates adjusted according to model sensitivity:

• ResNet50: 1 x 10^{-5}

• MobileNet: 1 x 10^{-4}

To reduce overfitting, both architectures incorporated L2 regularization and Dropout, though with different configurations.

• ResNet50: two Dropout layers with a rate of 0.5, Dense layers regularized with 5 x $10^{-4}$and the output layer with 1 x 10^{-3}

• MobileNet: one Dropout layer with a rate of 0.4, Dense layer regularized with 1 x 10^{-4}, and the output layer with 1 x 10^{-3}

A key distinction between the two setups was in model fine-tuning strategy.

• ResNet50 was fully fine-tuned, with all layers set to trainable.

• MobileNet followed a partial fine-tuning strategy, where the majority of layers were frozen and only the last 40 layers were trainable, following model compression recommendations.

Additionally, Early Stopping and Model Checkpointing were applied exclusively in the MobileNet experiment to improve generalization and automatically save the best-performing weights based on validation accuracy.

This configuration ensured that both models were trained consistently while respecting the structural requirements of each architecture.

4.7 Evaluation Metrics

To evaluate the performance of both ResNet50 and MobileNet, four standard classification metrics were used: Accuracy, Precision, Recall, and F1-Score. These metrics provide a comprehensive understanding of the models' effectiveness, especially in the presence of class imbalance within the dataset.

The metrics are defined mathematically as follows:

Accuracy

$$\text{Accuracy} = \frac{\boldsymbol{TP + TN}}{\boldsymbol{TP + TN + FP + FN}}$$

Precision

$$\text{Precision} = \frac{\boldsymbol{TP}}{\boldsymbol{TP + FP}}$$

Recall

$$\text{Recall} = \frac{\boldsymbol{TP}}{\boldsymbol{TP + FN}}$$

F1-Score

$$\text{F1 Score} = \mathbf{2} \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where:
TP = True Positives,
TN = True Negatives,
FP = False Positives,
FN = False Negatives.

Using these metrics allowed for a balanced assessment of classification performance for both models, capturing not only correctness (accuracy) but also sensitivity and robustness in detecting each class.

4.8 Results

Metric	ResNet50	MobileNet
Test Accuracy	**0.8151**	**0.8842**
Test Loss	**0.3842**	**0.2922**
Best Val Accuracy	**0.8888**	**0.8888**

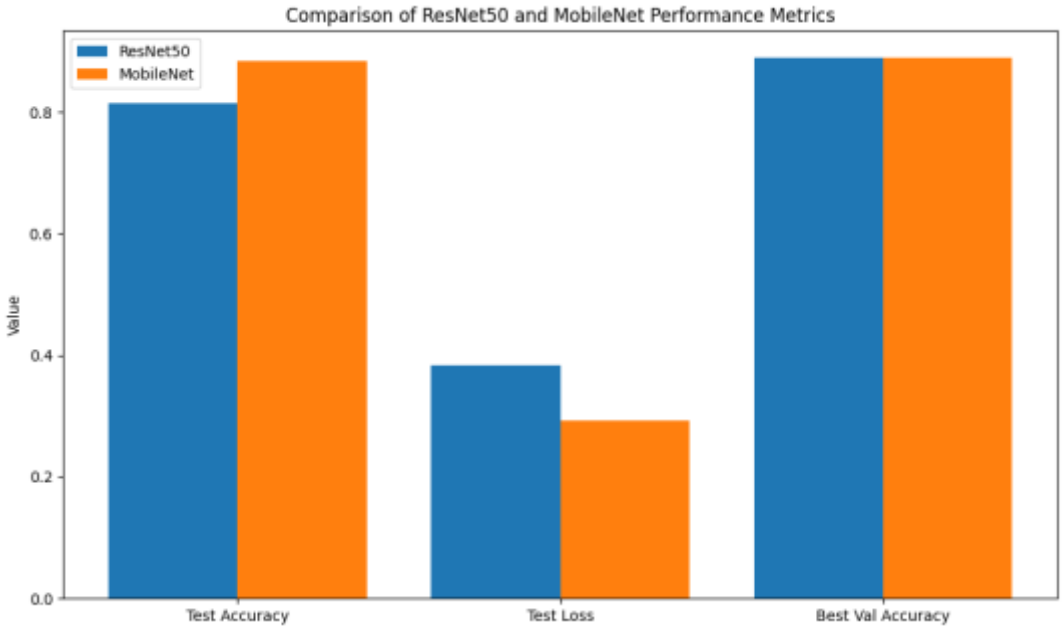


Figure 4.7: Comparison of ResNet50 and MobileNet Performance Metrics

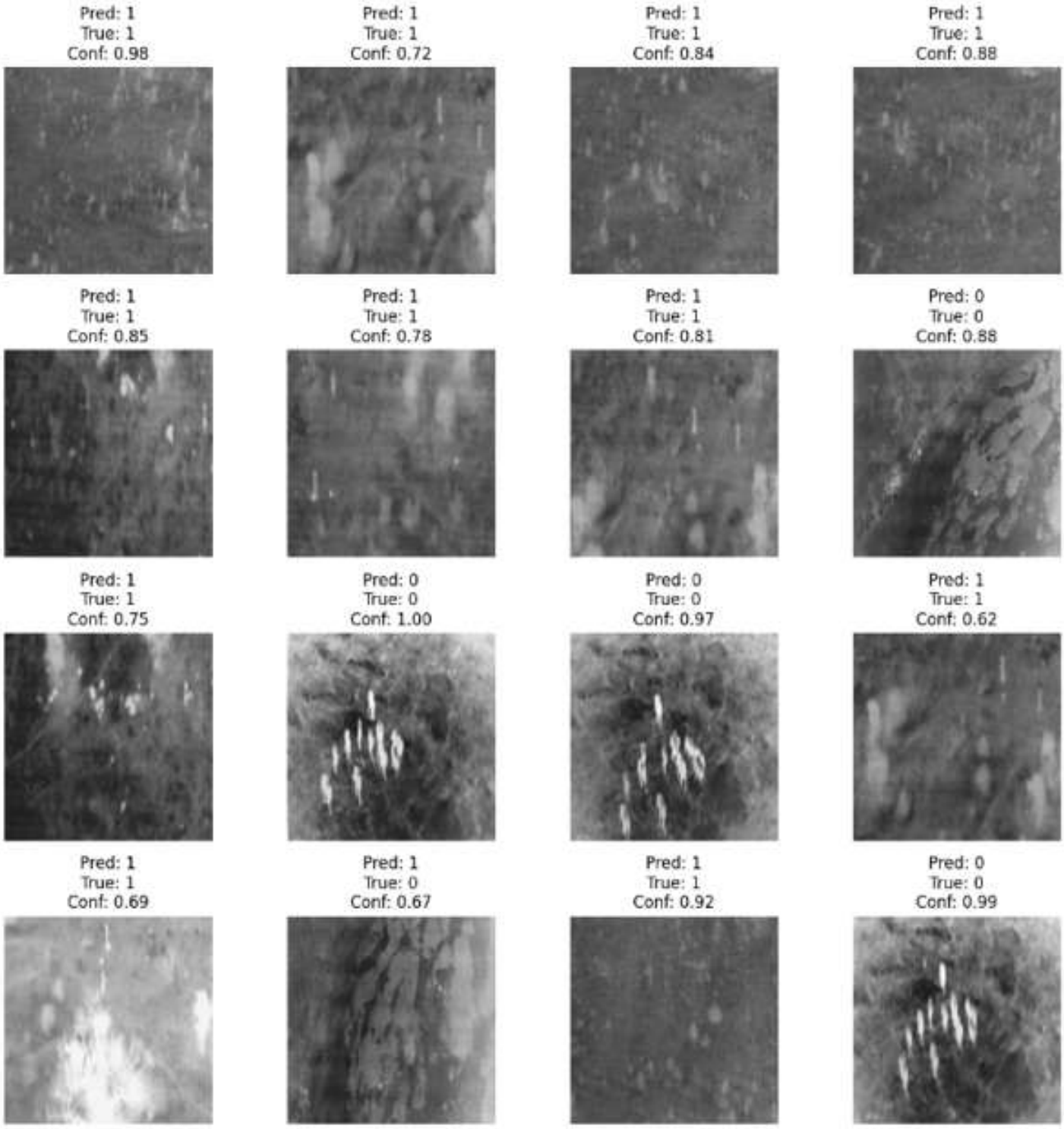


Figure 4.8: MobileNet Predictions

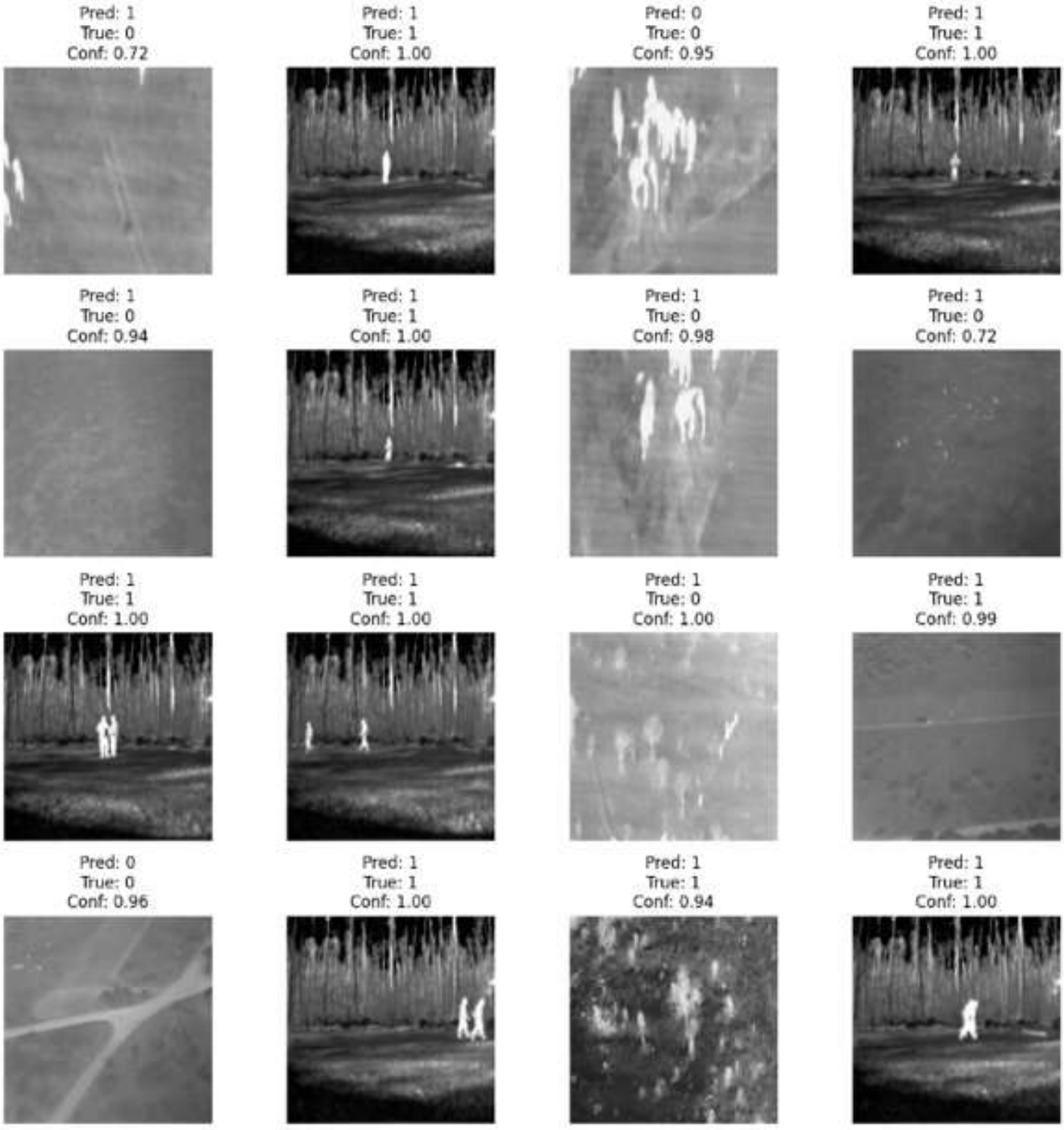


Figure 4.9: ResNet50 Predictions

4.9 Discussion

4.9.1 Model Characteristics

ResNet50, with its deep architecture, demonstrated strong feature extraction capabilities, while MobileNet provided efficient performance with lower computational requirements, establishing a practical reference for subsequent experiments.

4.9.2 Class Performance

Both models effectively distinguished between Human and Animal classes, offering insights into feature representation and training dynamics. These results serve as a clear baseline before introducing more advanced detection models.

4.9.3 Architectural Comparison

ResNet50 offers higher representational capacity suitable for capturing complex features.

MobileNet balances accuracy with efficiency, making it relevant for resource-aware scenarios.

4.9.4 Future Enhancements

These initial results highlight opportunities for refinement, such as data augmentation and optimized training strategies, which can guide improvements in upcoming YOLO-based experiments.

4.9.5 Practical Usage

MobileNet is advantageous where speed and efficiency are critical. ResNet50 provides a strong benchmark for accuracy-focused applications. These insights inform the choice of architecture and settings for the next phase with YOLO.

4.10 Summary

4.10.1 Overall Performance

Both ResNet50 and MobileNet successfully classified images into Human and Animal categories, providing a reliable reference point for more advanced object detection models.

4.10.2 Model Strengths

ResNet50 delivers high representational power suitable for comprehensive feature learning.

MobileNet achieves competitive performance with faster training and reduced computational demand.

Chapter 5: Human vs Anamel Detection

5.1 Dataset Enhancement and Expansion

One of the primary challenges identified at the end of Phase 1 was the limited quantity and diversity of training data. The dataset was therefore expanded before any YOLO training began to ensure the model would be exposed to a broader range of visual conditions and object appearances.

5.1.1 BirdsAI Dataset Expansion

To improve diversity, additional sequences were selected from the BirdsAI dataset that had not been included in the Phase 1 dataset. These sequences were specifically chosen because they featured animals captured at closer ranges and lower drone altitudes, resulting in larger and more visually detailed thermal signatures compared to the high-altitude footage used previously. This distinction was important because the earlier BirdsAI sequences predominantly showed animals as very small objects within wide-area aerial views, which limited the variety of thermal patterns available for training.

The newly added sequences from BirdsAI test set introduced animals that occupied a larger portion of the frame, providing the model with richer feature information about animal body shapes, thermal distribution, and movement characteristics. This expansion contributed meaningfully to the overall diversity of the dataset by supplementing the existing high-altitude animal footage with closer-range examples, making the combined dataset more representative of the range of conditions the system may encounter in real-world deployment.

All added sequences underwent the same cleaning, annotation verification, and format conversion pipeline described in Chapter 3, and were integrated into the unified dataset structure before the corrected frame-based splitting procedure described in Section 5.2 was applied.

File name	Number of images after cleaning
12	117
55	30
58	162
60	199
65	304
349	167
352	166
353	400
357	166
358	108
361	395
362	240
371	1702
Total	**4,156**

5.2 Frame-Based Dataset Splitting

A critical methodological improvement implemented in Phase 2 of our project was the complete redesign of the dataset splitting strategy. In Phase 1, the train/validation/test split was performed at the image level using a random shuffle across all images in the merged dataset. While this approach is standard practice for datasets composed of independent, identically distributed samples, it introduced a fundamental and serious problem when applied to video-derived data (temporal data leakage). This section documents the nature of the problem, how it was detected, and the full process by which it was corrected.

5.2.1 Background: How the Dataset Was Constructed

To fully understand why temporal leakage was such a critical problem for this project, it is necessary to recall how the underlying dataset was assembled. As described in Chapter 3, the dataset was built from multiple sources, the majority of which were originally recorded as continuous thermal video sequences rather than isolated images. The OSU Thermal Pedestrian Dataset consists of thermal video clips recorded by a static ground camera, from which individual frames were extracted. The Terravic Motion IR Dataset similarly provides long sequences of outdoor surveillance footage, with some sequences running to over 1,300 frames each. The BirdsAI dataset provides continuous aerial video of animal herds from drone flights, with single sequences reaching several thousand frames. Even the synthetic KSA SDS dataset was generated by flying a simulated drone along continuous paths through the environment, producing temporally ordered sequences.

In all of these cases, consecutive frames share an enormous amount of visual information. The background scene is identical, the camera position changes only slightly between frames, and any objects present in one frame are almost certainly present in the adjacent frames in very similar positions and orientations. This is a fundamental characteristic of video data: temporal correlation between neighboring frames is extremely high, and the degree of similarity decreases only gradually as the temporal distance between frames increases.

5.2.2 The Problem of Temporal Leakage

Temporal data leakage occurs when information that should be exclusively available in the training set is inadvertently also present in the validation or test set, causing evaluation metrics to appear better than they would in a genuine real-world deployment scenario. In the context of video-derived image datasets, this form of leakage arises directly from random image-level splitting [16].

When all frames from all video sequences are pooled together into a single flat list and then randomly divided into training, validation, and test subsets, frames from the same video sequence become distributed across all splits. As a result, consecutive or highly similar frames from the same scene may appear simultaneously in both the training and testing sets. Since adjacent frames in thermal videos differ only slightly, the model can unintentionally memorize scene-specific patterns instead of learning generalized detection features.

To illustrate this issue, the Terravic irw07 sequence contains 1067 frames captured in an outdoor thermal environment with human subjects walking through a wooded scene, as shown in Figure 5.1. When random image-level splitting is applied, frames from this single sequence become shared across the training, validation, and testing subsets. Consequently, if a model observes one frame during training, it can easily recognize nearby frames appearing in the test set because consecutive frames remain visually almost identical. The background texture, thermal noise pattern, camera position, and even the subject's thermal signature remain highly consistent across adjacent frames. Therefore, the model does not truly generalize to unseen data; instead, it benefits from memorizing the visual characteristics of previously seen scenes.

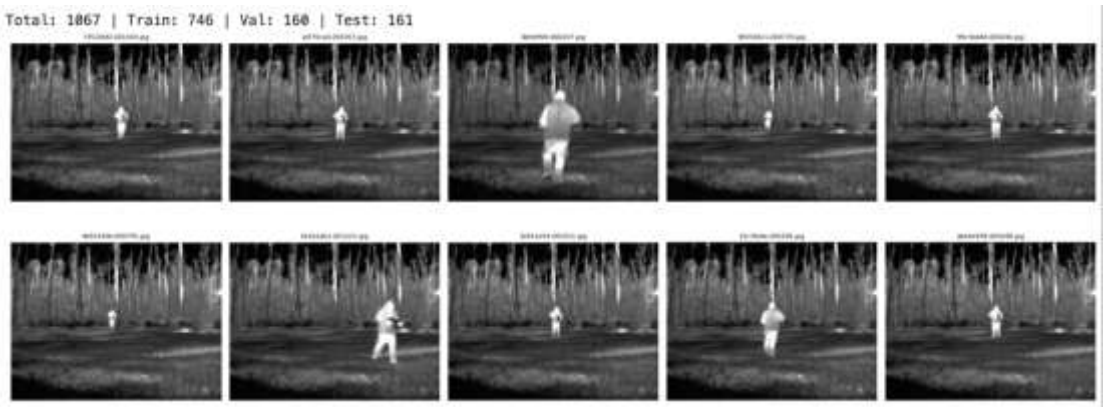


Figure 5.1: Sample Frames and Split Distribution from the Terravic irw07 After Splitting

This same problem, and even more so, applies to the BirdsAI dataset, where some clips contain over 6,000 frames of continuous drone footage over a single herd of animals. Randomly splitting this single clip would place thousands of nearly identical frames in both the training and test sets simultaneously.

As a result of this leakage, the reported evaluation metrics are systematically and significantly inflated. The model appears to generalize well because the test set is not actually testing generalization, but rather the model's ability to recognize scenes it has seen before in a slightly different time frame. This was the precise explanation for the unusually high metrics observed in our first YOLO experiment on the older dataset (F1 = 98.96%, mAP@0.50 = 99.42%), which were initially encouraging but later proved to be due to leakage rather than genuine detection capability.

5.2.3 Detection and Confirmation of the Leakage Problem

The leakage problem was formally identified and confirmed during the early YOLO training experiments in Phase 2. The first strong signal was the observation that YOLOv8m trained on the old image-level split dataset achieved a near-perfect F1 score of 98.96% and mAP@0.50 of 99.42%, which is extremely unusual for a two-class thermal object detection problem given the known challenges of thermal imagery (low contrast, weak textures, thermal noise, and scale variation). Hence, the correction of this problem became the highest-priority task before any further model training could be considered meaningful.

5.2.4 Implementation of the Corrected Split

Correcting the splitting strategy was not a single one-step fix. It was an iterative process driven by the goal of improving model performance, where each training run provided feedback that informed the next adjustment. The core principle that remained constant throughout was that each source video sequence must reside entirely within one split, so that no frames from the same sequence could appear in different subsets. The process involved a combination of manual inspection and Python scripting, applied across several refinement cycles until a well-balanced and representative split was achieved.

At various stages of the process, both automated scripts and manual file operations were used to adjust the composition of the validation and test sets, with the objective of ensuring that both splits contained a diverse and representative mix of scenes, environments, and class instances. This process confirmed that careful, leakage-free splitting is as important as model selection and hyperparameter tuning in producing reliable and meaningful evaluation results.

5.3 Enhanced Dataset Statistics and Visualization

Following the dataset expansion and the implementation of the corrected frame-based splitting strategy described in Sections 5.1 and 5.2, the final unified dataset consists of 26,799 images distributed across three splits.

Split	**Image Count**
Train	20,507
Val	3,169
Test	3,123
Total	**26,799**

Table 5.3: Final Dataset Split Summary

The dataset was split using the frame-based splitting strategy to ensure no temporal leakage between splits.

The following table presents the per-class image distribution across all splits. An image is counted for a class if it contains at least one annotated object of that class.

Split	**Animal Images**	**Human Images**	**Total Images**
Train	9,861	11,494	20,507
Val	2,058	1,111	3,169
Test	2,056	1,067	3,123
Total	**13,975**	**13,672**	**26,799**

Table 5.4: Image Count per Class per Split

As shown in the figures below, the overall dataset is well balanced between the two classes, with animal images accounting for 50.6% of the total and human images accounting for 49.4%. This near-equal distribution is a direct result of the dataset expansion carried out in Phase 2, which added closer-range BirdsAI animal sequences to supplement the originally human-dominant real-world data. The balance between classes is expected to support more stable and unbiased model training, preventing the model from developing a systematic preference toward either class.

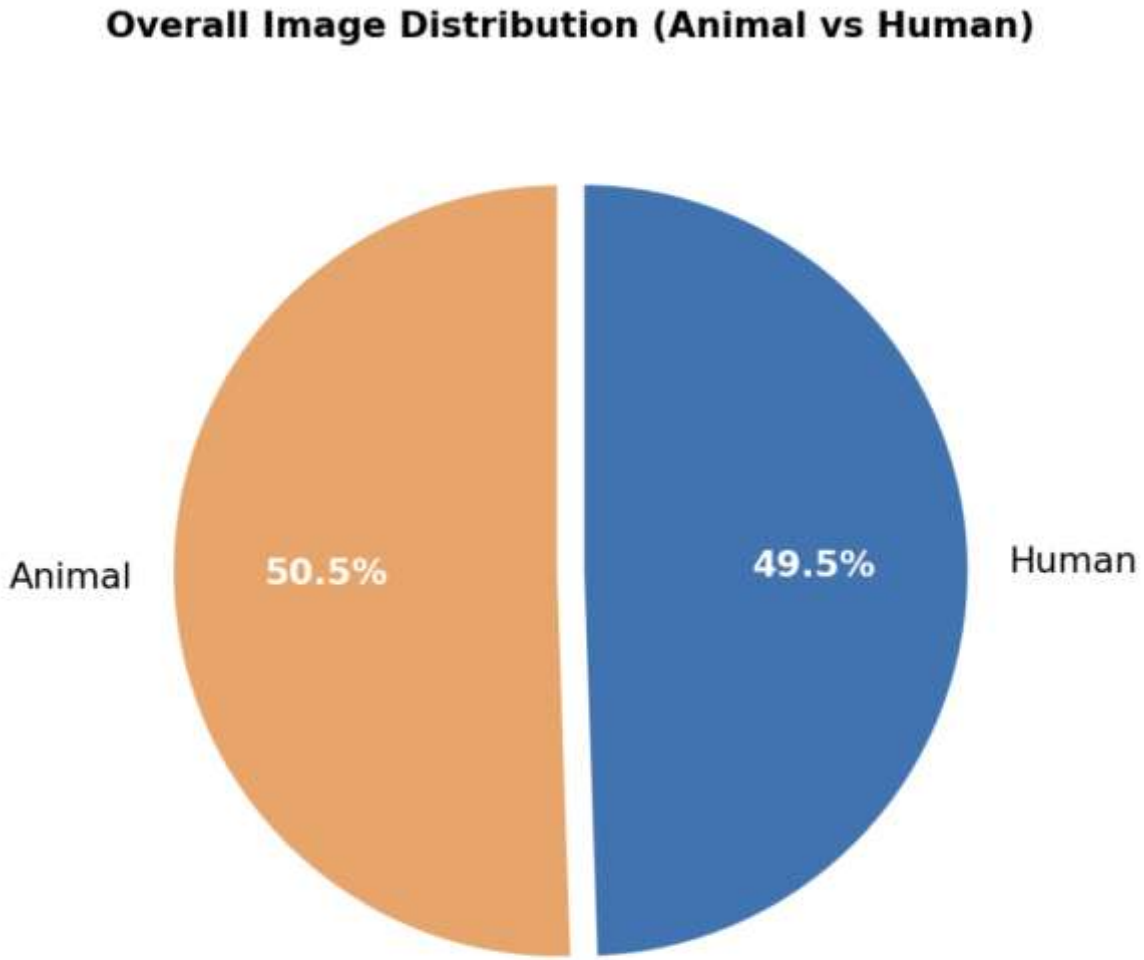


Figure 5.2: Overall Image Distribution (Animal vs Human) in Final Dataset

The bar chart in Figure 5.3 shows the per-class image counts broken down by split. Across all three splits, the animal and human classes maintain a broadly consistent ratio, confirming that the frame-based splitting procedure distributed both classes proportionally without introducing significant class imbalance in any individual subset. The training set, which contains 20,507 images, provides a sufficiently large and diverse pool for the YOLO26m model to learn robust detection features, while the validation and test sets, with approximately 3,100 to 3,200 images each, offer representative and independent evaluation subsets.

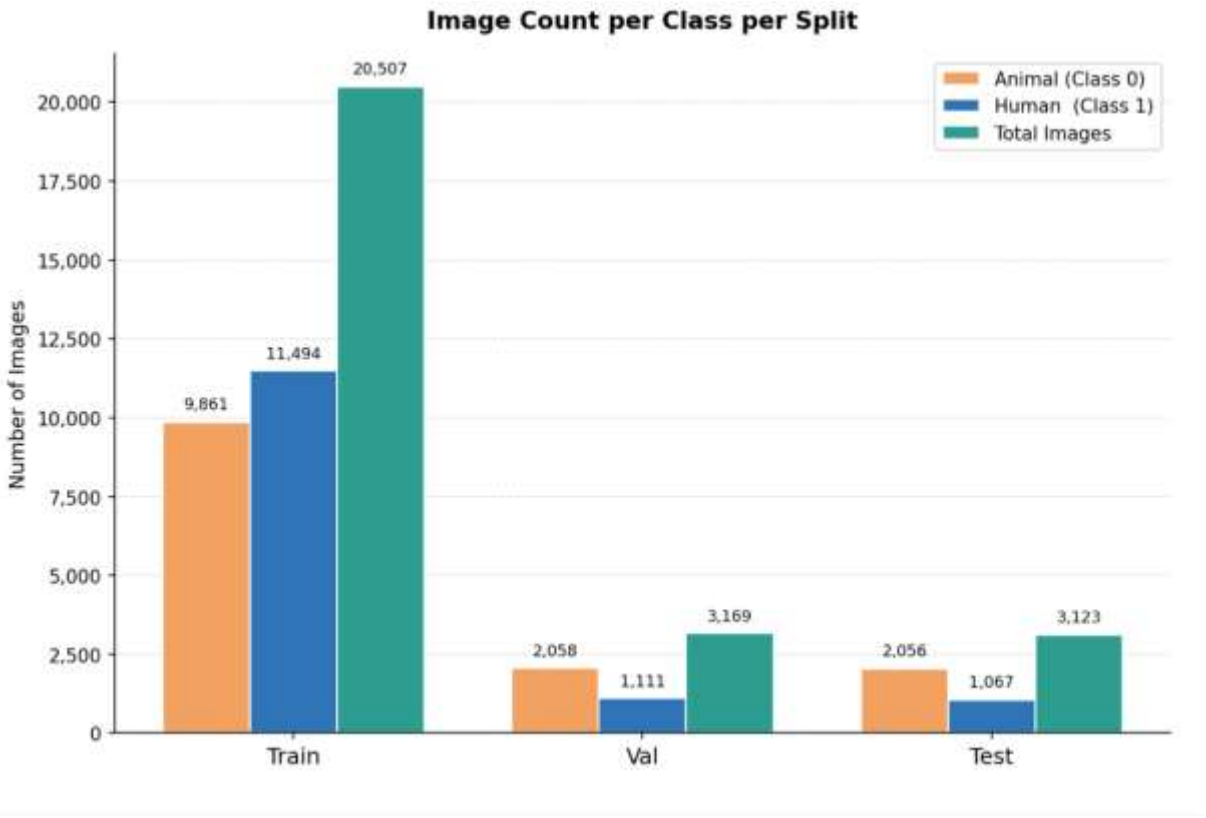


Figure 5.3: Image Count per Class per Split

5.4 Last YOLO Model Training and Configurations

This section delves into the granular details of our finalized YOLO model, offering an in-depth analysis of its structural configuration, the trained weights obtained, and the diverse set of parameters that governed the model's learning process and final performance.

5.4.1 Model Selection: YOLO26m

The YOLO26m model, the medium-sized variant of the YOLO26 generation, was selected as the final detection architecture for this project. YOLO26 represents a significant advancement over earlier YOLO generations in terms of feature extraction depth, detection head design, and overall detection accuracy, while still maintaining the inference speed that makes YOLO architectures well-suited for real-time drone surveillance applications. The medium-sized variant was chosen specifically because it strikes an appropriate balance between representational capacity and computational efficiency: it is powerful enough to handle the challenges of thermal imagery detection, yet compact enough to be a realistic candidate for deployment on edge hardware in future phases of the project.

The YOLO26m architecture consists of 280 layers with approximately 21 million trainable parameters and a computational cost of 74.7 GFLOPs. Its backbone employs a series of convolutional layers with progressively increasing receptive fields, enabling the model to capture both fine-grained local features such as the edges and shapes of small thermal signatures, and broader contextual patterns that help distinguish human silhouettes from animal body shapes in cluttered thermal scenes. The detection head operates at multiple scales simultaneously, which is particularly important for this dataset since objects can appear at very different sizes depending on the drone’s altitude at the time of capture.

5.4.2 Training Configuration

The YOLO26m model was trained using the Ultralytics framework on Google Colab with an NVIDIA L4 GPU (22 GB VRAM), which provided sufficient memory for processing the full dataset at the selected image resolution. The training was configured with a maximum of 50 epochs and a batch size of 16 images. An early stopping mechanism with a patience value of 15 epochs was applied to prevent overfitting and avoid unnecessary computation once performance plateaued.

The input image resolution was set to 640×640 pixels, which is the standard resolution for YOLO-based detection tasks and provides a suitable balance between spatial detail and inference speed. The optimizer was set to automatic selection (auto), allowing the framework to choose the most appropriate optimizer based on the model architecture. The initial learning rate (lr0) was set to 0.01, with a final learning rate factor (lrf) of 0.01, guiding the learning rate schedule over the course of training. Momentum was set to 0.937 and weight decay to 0.0005 to support stable convergence and regularization.

Regarding data augmentation, the standard Ultralytics augmentation pipeline was applied during training. This included horizontal flipping with a probability of 0.5, mosaic augmentation enabled at a scale of 1.0, HSV color space jitter (hue: 0.015, saturation: 0.7, value: 0.4), random erasing with a probability of 0.4, and a random scale factor of 0.5. Mosaic augmentation was automatically disabled during the final 10 epochs (close_mosaic=10) to allow the model to stabilize on clean, unaugmented images before the end of training. These augmentations were applied to help the model generalize across variations in thermal contrast, object scale, and scene composition.

The model was initialized with pretrained weights from the YOLO26m ImageNet checkpoint and fine-tuned end-to-end on our two-class thermal dataset. Training was conducted with Automatic Mixed Precision (AMP) enabled to reduce memory usage and improve computational efficiency without affecting model accuracy. The training process was designed to be resumable, with checkpoints saved after every epoch to Google Drive to prevent data loss in the event of session interruptions.

Parameter	**Value**
Model	YOLO26m
Pretrained	Yes (ImageNet)
Epochs	50 (early stop at epoch 30)
Batch Size	16
Image Size	640 × 640
Optimizer	Auto
Initial Learning Rate (lr0)	0.01
Final Learning Rate (lrf)	0.01
Momentum	0.937
Weight Decay	0.0005
Early Stopping Patience	15
Mosaic Augmentation	1.0 (disabled last 10 epochs)
Horizontal Flip Probability	0.5
Random Erasing	0.4
AMP	Enabled
GPU	NVIDIA L4 (22 GB)

Table 5.1: YOLO26m Training Hyperparameters

5.5 Final Model Evaluation

5.5.1 Evaluation Setup

The final model was evaluated using both validation and test datasets to assess its detection performance and generalization capability. The evaluation was based on standard object detection metrics, including Precision, Recall, F1-score, mAP@50, and mAP@50–95.

The model was trained using the YOLO26m architecture for up to 50 epochs. However, early stopping was triggered at epoch 30 due to the lack of further improvement, indicating that the model reached a stable convergence point without overfitting.

5.5.2 Overall Performance

The final YOLO26m model was evaluated on both the validation and test splits of the corrected frame-based dataset.

Dataset	Precision	Recall	F1-Score	mAP@50	mAP@50–95
Validation	0.8959	0.7899	0.8396	0.8734	0.5282
Test	0.9448	0.8381	0.8883	0.8818	0.5681

Table 5.2: Performance Comparison on Validation and Test Datasets

The model achieved strong and consistent performance across both datasets. The mAP@50 exceeded 0.87 on validation and 0.88 on the test set, indicating reliable object detection accuracy at a standard IoU threshold. The test set showed slightly higher precision and F1-score than the validation set, which suggests that the model generalizes well to unseen data and does not overfit to the validation distribution. Recall values are somewhat lower than precision on both splits, indicating that while the majority of detections are correct, a small proportion of objects particularly in complex or cluttered scenes may be missed. The mAP@50–95 values of 0.5282 and 0.5681 reflect the stricter multi-threshold evaluation criteria and indicate that localization accuracy at tighter IoU thresholds remains an area for future improvement.

5.5.3 Training Convergence

The training process showed stable convergence, with loss values decreasing steadily and detection metrics improving consistently across epochs. Early stopping was triggered at epoch 30, before the maximum of 50 epochs was reached, indicating that the model reached its optimal performance point without requiring the full training budget. This is consistent with well-configured hyperparameters and an appropriate learning rate schedule. The use of model checkpointing ensured that the best-performing weights were preserved throughout training. The convergence behavior reflects a well-balanced training configuration in which the model learned effectively from the corrected dataset without signs of overfitting or instability.

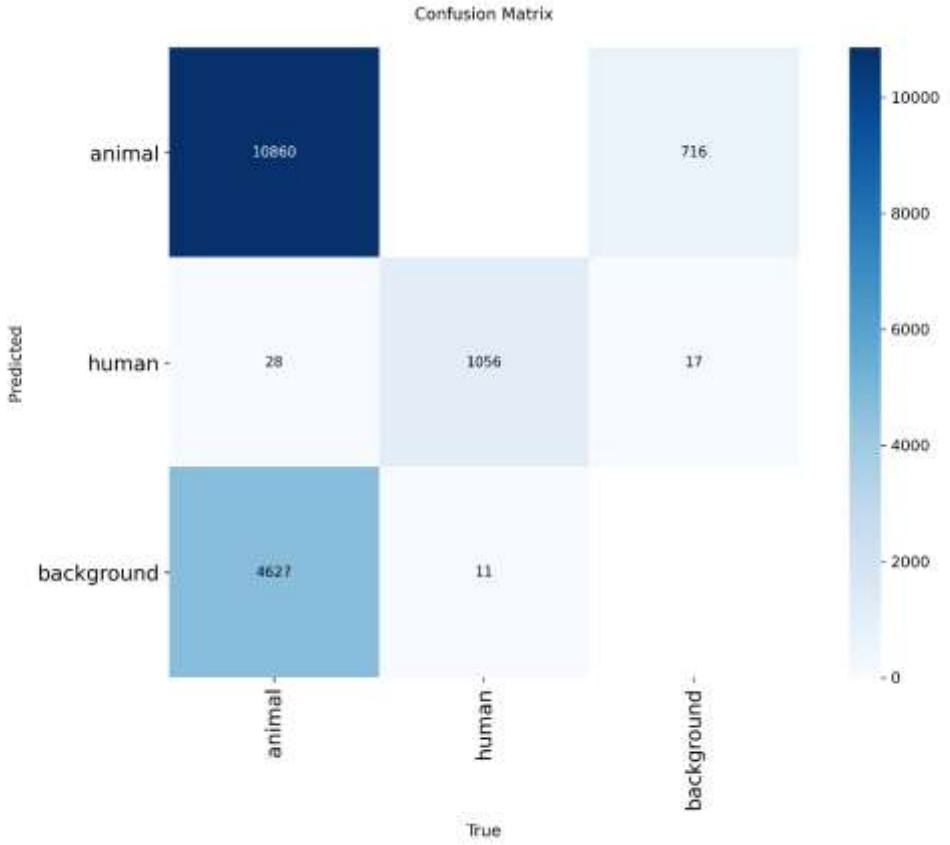


Figure 5.4: Confusion Matrix Showing Classification Results for Animal and Human

5.5.4 Class-wise Performance

A breakdown of per-class performance on the validation set revealed meaningful differences between the two target classes. The animal class, which contained a substantially larger number of instances (17,430 instances across 2,058 images), achieved a precision of 0.898 and a recall of 0.589, with an mAP@50 of 0.765. The human class, despite having fewer instances (1,631 instances across 1,111 images), achieved notably higher recall (0.991) and mAP@50 (0.981), with a precision of 0.893.

The lower recall for the animal class can be attributed to the inherent variability in animal thermal signatures. Animals differ widely in body size, shape, and heat distribution depending on species, posture, and proximity to the camera. At higher drone altitudes, smaller animals may produce thermal signatures that are difficult to distinguish from background noise. In contrast, the human class benefits from a more consistent upright silhouette and predictable heat distribution patterns, which makes it easier for the model to detect reliably across different scenes.

Despite this disparity, both classes demonstrated strong precision values above 0.89, confirming that when the model does produce a detection, it is highly likely to be correct. These results support the practical effectiveness of the system for real-world deployment, where false alarm minimization is a key operational requirement in border surveillance scenarios.

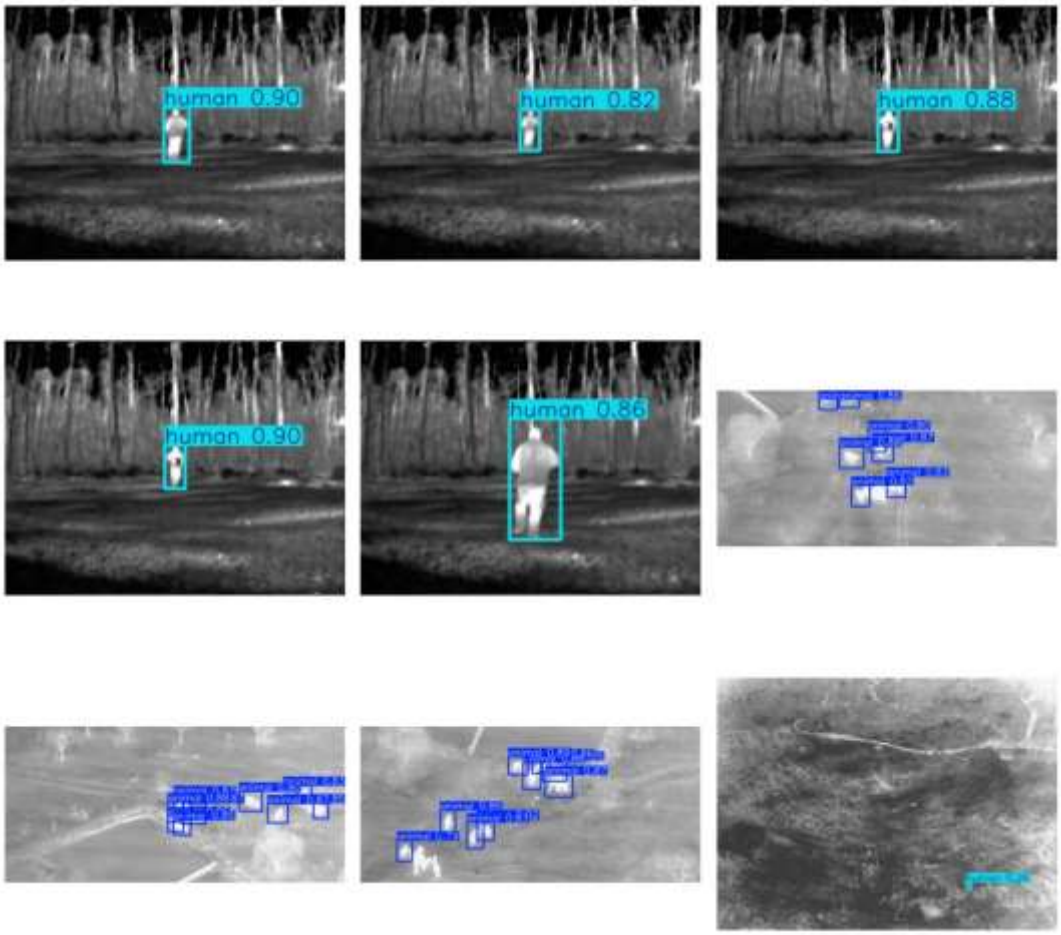


Figure 5.5: YOLO detection results showing humans and animals with bounding boxes and confidence scores

Chapter 6: Summary and Future Work

6.1 Summary

This project proposed and developed a deep learning-based object detection system for border surveillance and search and rescue operations using thermal infrared imagery captured by unmanned aerial vehicles. The system was designed to address the critical challenge of automatically distinguishing between humans and animals under low-visibility and nighttime conditions, where traditional camera-based surveillance systems fail to provide reliable performance.

The work was carried out across two phases. In the first phase, the foundation of the system was established through dataset collection, cleaning, and preparation, as well as initial classification experiments using ResNet50 and MobileNet. These experiments confirmed the feasibility of the approach and provided a clear baseline for model selection and preprocessing strategy. The results demonstrated that CNN-based architectures could classify thermal images into the human and animal categories with high accuracy, motivating the transition to full object detection in the second phase.

The second phase addressed several significant challenges identified during the first phase. A critical methodological problem temporal data leakage caused by random image-level splitting of video-derived data was identified and corrected through the implementation of a strict frame-based splitting strategy. This correction, while reducing the apparent performance metrics from the inflated values observed in initial YOLO experiments, produced a more realistic and trustworthy evaluation framework that reflects genuine model generalization.

The dataset was also expanded in the second phase to improve class diversity, through the addition of BirdsAI sequences featuring closer-range animal footage. The final unified dataset comprised over 26,000 images spanning real thermal footage and synthetically generated desert scenes, split into training, validation, and test sets with no temporal leakage between splits.

The YOLO26m architecture was selected as the final detection model based on its balance of accuracy, inference speed, and suitability for edge deployment. After training for 30 effective epochs with early stopping, the model achieved an mAP@50 of 0.8734 on the validation set and 0.8818 on the test set, with F1-scores of 0.8396 and 0.8883 respectively. These results demonstrate that the system can reliably detect and classify humans and animals in diverse thermal environments, providing a strong foundation for the next phase of the project.

6.2 Future work

Several directions for future work have been identified based on the results and limitations observed across both phases of the project.

The most immediate priority is the integration of the trained YOLO26m model into a real-time drone surveillance pipeline. This would involve deploying the model on embedded edge hardware mounted on a UAV platform and validating its detection performance under actual flight conditions, including varying altitudes, camera angles, and environmental factors such as wind-induced vibration and thermal crossover during sunrise and sunset periods.

Data collection from real Saudi desert environments remains a critical long-term goal. The current dataset lacks real thermal footage from the Rub' al Khali region, meaning the model has not been directly exposed to the specific terrain textures, ambient temperature profiles, and wildlife species native to the operational environment. Collecting even a modest number of real desert sequences and incorporating them into training is expected to significantly improve model generalization in this domain.

Expanding the synthetic dataset using the AirSim simulation environment is also planned. Future synthetic data generation will incorporate additional environmental conditions such as sandstorm effects, fog, and varying time-of-day temperature distributions. A wider variety of animal species representative of the Arabian Peninsula including camels, Arabian oryx, and sand gazelles will also be modeled to improve the system's coverage of realistic threat and non-threat scenarios.

Finally, a comprehensive field evaluation involving collaboration with relevant security and emergency response authorities is planned to assess the practical utility of the system under operational conditions and to identify any additional requirements for real-world deployment.

Acknowledgement

The authors would like to express their sincere gratitude to their project supervisor, Dr. Saeed Alahmari, for his continuous guidance, valuable feedback, and dedicated support throughout both phases of this project. His expertise and constructive direction were instrumental in shaping the methodology and

References

[1] Mittal, U., Srivastava, S. and Chawla, P. (2019) 'Object detection and classification from thermal images using region-based convolutional neural network', Journal of Computer Science, 15(7), pp. 961-971. DOI: 10.3844/jcssp.2019.961.971.

[2] *upGrad, "Basic CNN Architecture: A Detailed Explanation of the 5 Layers in Convolutional Neural Networks."*
Available: https://www.upgrad.com/blog/basic-cnn-architecture

[3] LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning', Nature, 521(7553), pp. 436–444.

[4] Bondi, E., Jain, R., Aggrawal, P., Anand, S., Hannaford, R., Kapoor, A., Piavis, J., Shah, S., Joppa, L., Dilkina, B. and Tambe, M., 2020. BIRDSAI: A dataset for detection and tracking in aerial thermal infrared videos. In *Proceedings of the IEEE/CVF Winter conference on applications of computer vision* (pp. 1747-1756).

[5] Murugesan, M. and Mahendiran, A., 2024, June. Animal conservation applications for wildlife detection and classification using various machine learning algorithms. In *AIP Conference Proceedings* (Vol. 3112, No. 1, p. 020028). AIP Publishing LLC.

[6] Backman, K., Wood, J., Brandimarti, M., Beranek, C.T. and Roff, A., 2025. Human inspired deep learning to locate and classify terrestrial and arboreal animals in thermal drone surveys. *Methods in Ecology and Evolution*.

[7] Zhang, L., Chen, X., Liu, J. and Yan, T., 2024, July. ASFN: An RGB-T Adaptive Selection Fusion Network for Nighttime Tracking. In *Proceedings of the 2024 International Conference on Intelligent Perception and Pattern Recognition* (pp. 111-120).

[8] Wang, W., Li, C., Zhang, D., Zhou, H., Xie, M., Zhou, H. and Fu, K., 2024. FcFNet: A Challenge-based Feature Complementary Fusion Network for RGBT Tracking. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*.

[9] Xue, Y., Zhang, J., Lin, Z., Li, C., Huo, B. and Zhang, Y., 2023. SiamCAF: Complementary attention fusion-based Siamese network for RGBT tracking. *Remote Sensing*, *15*(13), p.3252.

[10] Tan, H., Ou, D., Zhang, L., Shen, G., Li, X. and Ji, Y., 2022. Infrared sensation-based salient targets enhancement methods in low-visibility scenes. *Sensors*, *22*(15), p.5835.

[11] OTCBVS Benchmark Dataset Collection (n.d.) *OTCBVS Benchmark*. Available at: https://vcipl-okstate.org/pbvs/bench/index.html

[12] Microsoft AirSim (n.d.) *AirSim*. Available at: https://microsoft.github.io/AirSim/

[13] *He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp.770–778).*

[14] *Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H., 2017. MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.*

[15] *Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. & Fei-Fei, L. (2009) ImageNet: A Large-Scale Hierarchical Image Database. 2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 248–255.*

[16] Figueiredo, R.B.D. and Mendes, H.A. (2024**)** 'Analyzing information leakage on video object detection datasets by splitting images into clusters with high spatiotemporal correlation', *IEEE Access*, 12, pp. 47646–47655.